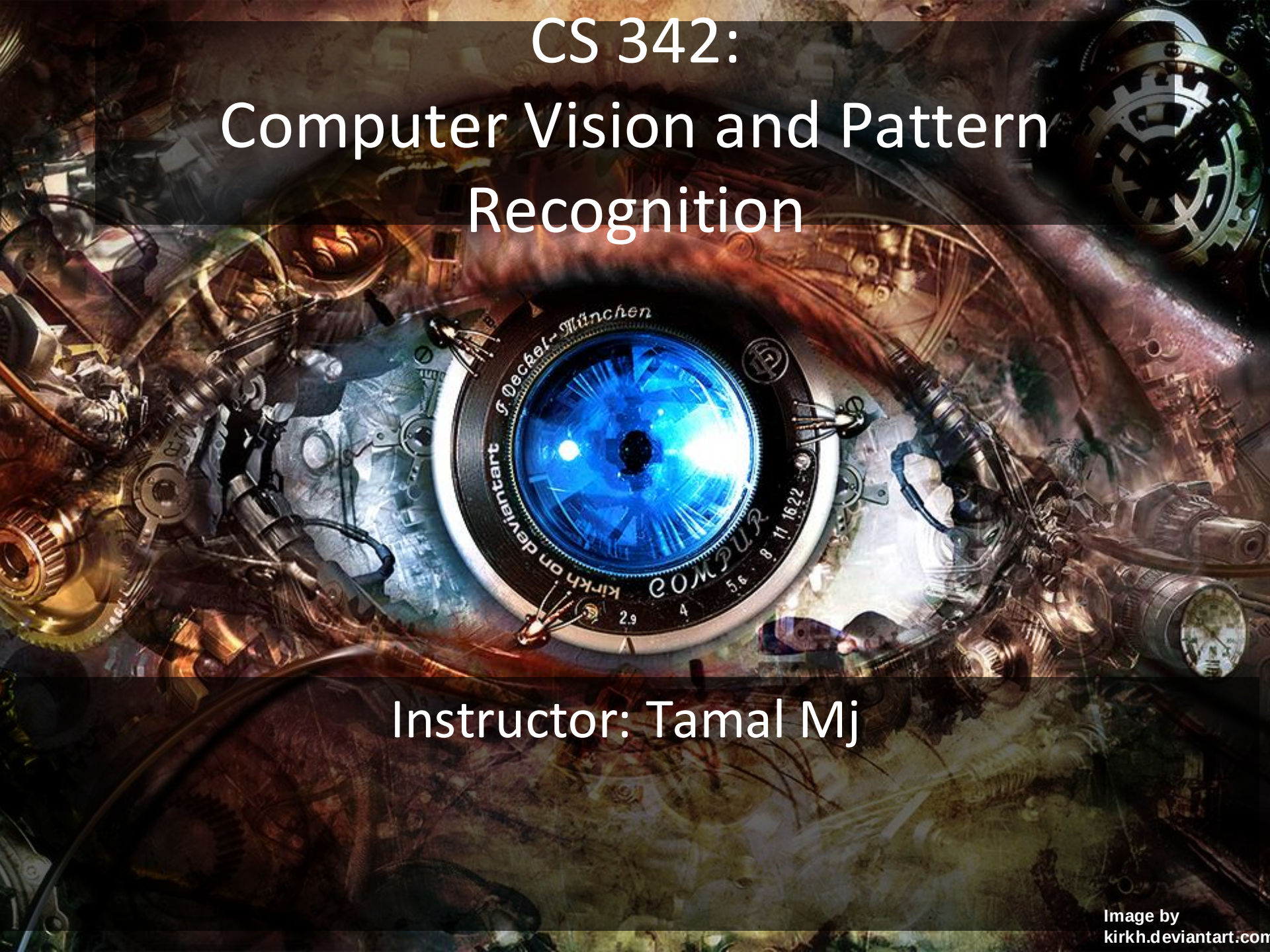
CS 342:
Computer Vision and Pattern Recognition



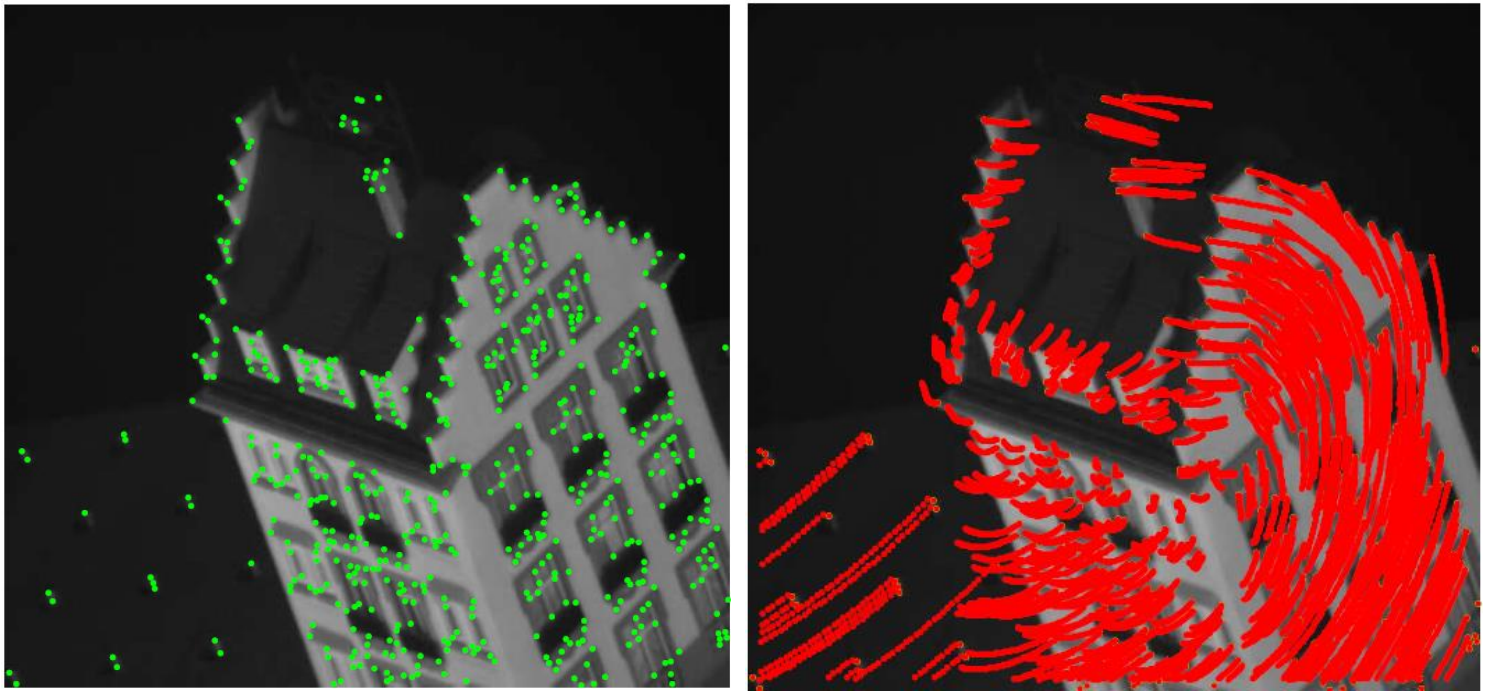
Instructor: Tamal Mj

Image by
kirkh.deviantart.com

Shanti Mantra

Lyrics in Sanskrit	Meaning in English
ॐ सह नाववतु।	Om, May we all be protected
सह नौ भुनक्तु।	May we all be nourished
सह वीर्यं करवावहै।	May we work together with great energy
तेजस्वि नावधीतमस्तु	May our intelect be sharpened (may our study be effective)
मा विद्विषावहै।	Let there be no Animosity amongst us
ॐ शान्तिः शान्तिः शान्तिः ॥	Om, peace (in me), peace (in nature), peace (in divine forces)

Feature Tracking and Optical Flow



Feature Tracking

This class: Recovering motion

- Feature tracking
  - Extract visual features (corners, textured areas) and "track" them over multiple frames

- Optical flow
  - Recover image motion at each pixel from spatio-temporal image brightness variations

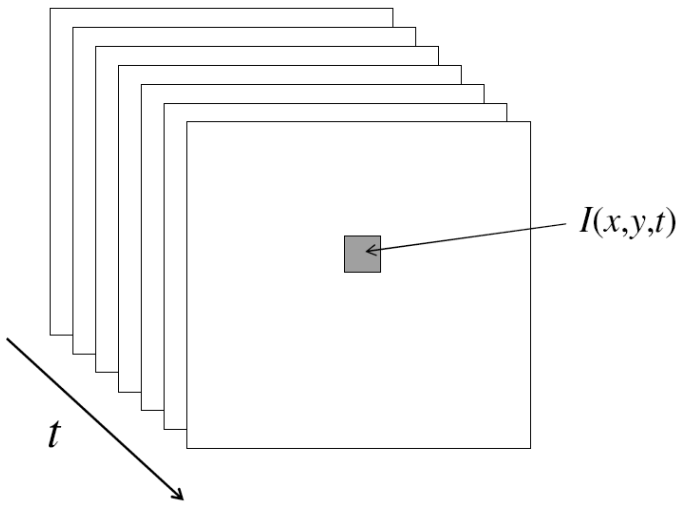
Two problems, one registration method

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence*, 1981.

Image registration means aligning two images by estimating the transformation between them.

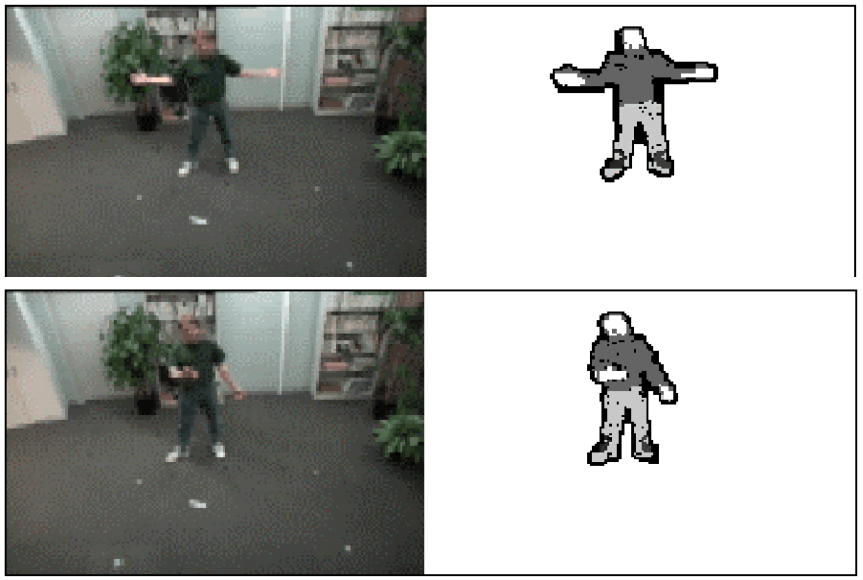
Video

- A video is a sequence of frames captured over time
- Now our image data is a function of space (x, y) and time (t)



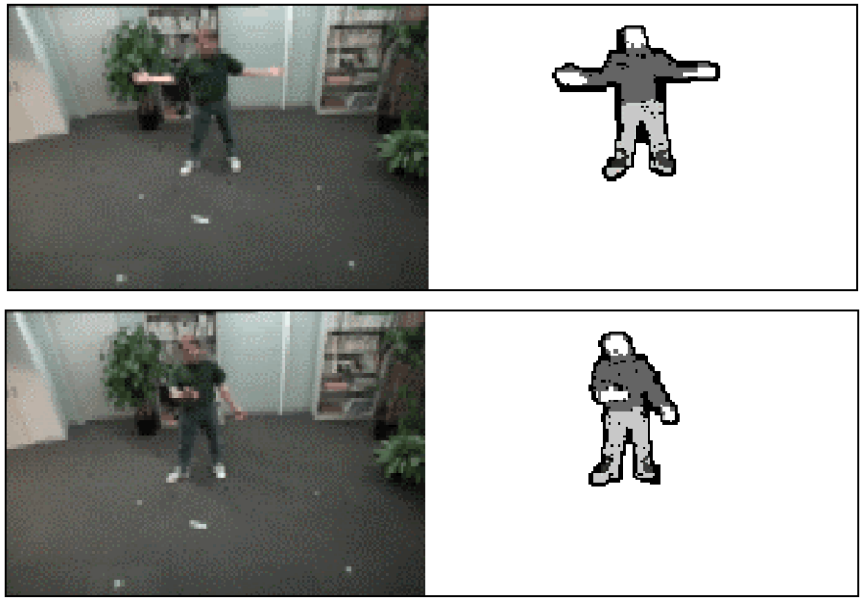
Motion Applications: Segmentation of video

- Background subtraction
  - A static camera is observing a scene
  - Goal: separate the static *background* from the moving *foreground*

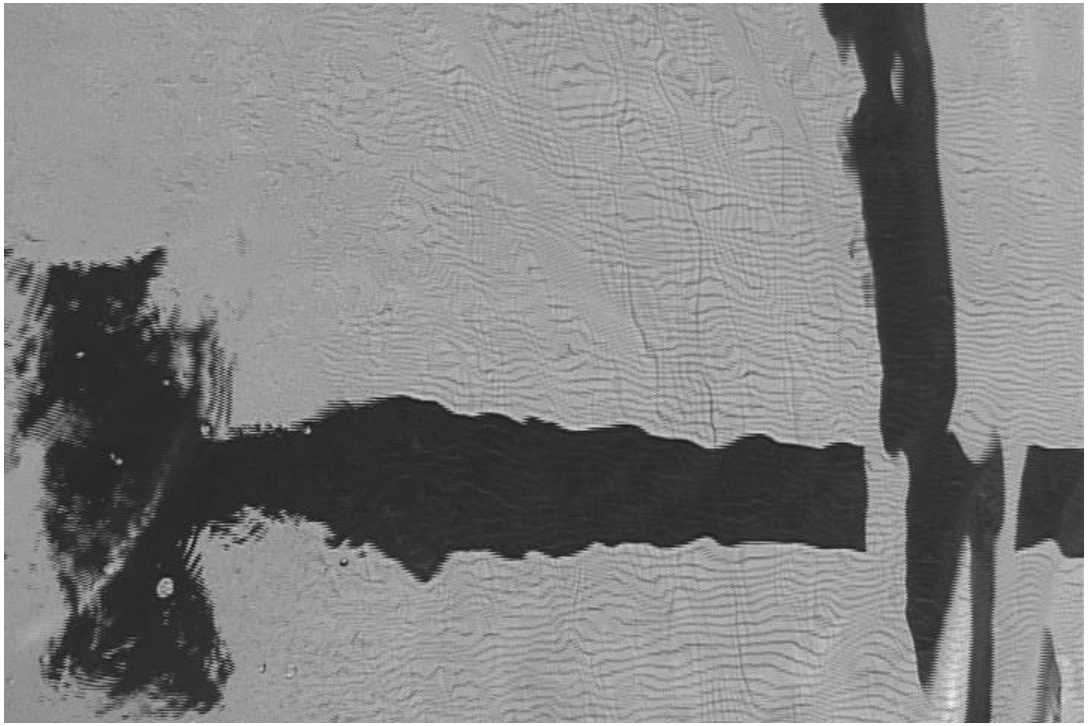


Motion Applications: Segmentation of video

- Background subtraction (Segmentation)
  - A static camera is observing a scene
  - Goal: separate the static *background* from the moving *foreground*

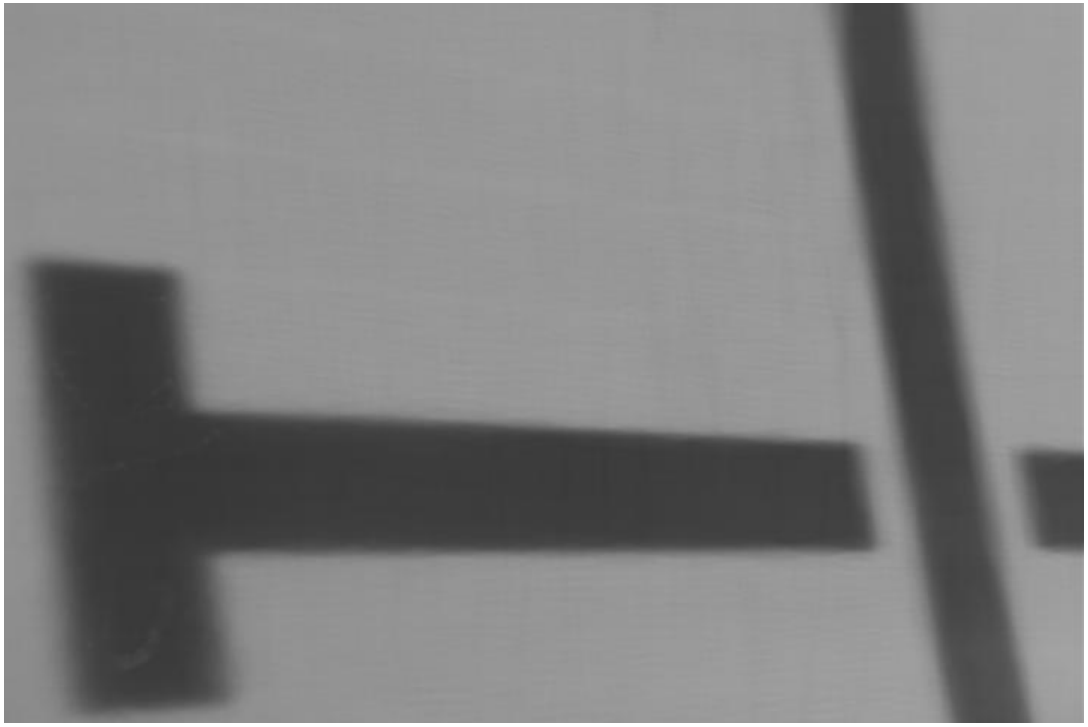


Input Video



Alyosha Efros, CMU

Average Image



Alyosha Efros, CMU

Background Subtraction

- Given an image (mostly likely to be a video frame), we want to identify the **foreground objects** in that image!

Motivation

- In most cases, objects are of interest, not the scene.
- Makes our life easier: less processing costs, and less room for error.

Slide credit: Birgi Tamersoy

Simple Approach

Image at time t:
$I(x, y, t)$

Background at time t:
$B(x, y, t)$

$$| \quad - \quad | > Th$$

1. Estimate the background for time t.
2. Subtract the estimated background from the input frame.
3. Apply a threshold, Th, to the absolute difference to get the **foreground mask**.

But, how can we estimate the background?

Frame Differencing

- Background is estimated to be the previous frame. Background subtraction equation then becomes:

$$B(x, y, t) = I(x, y, t-1)$$
$$\Downarrow$$
$$|I(x, y, t) - I(x, y, t-1)| > Th$$

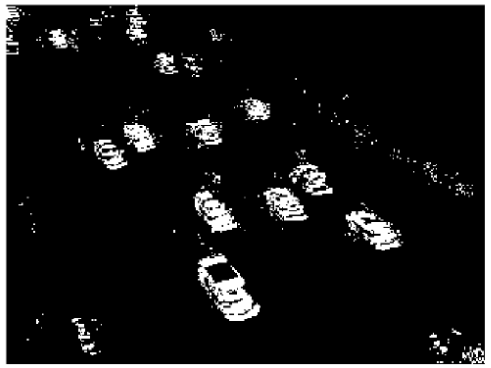
- Depending on the object structure, speed, frame rate and global threshold, this approach may or may **not** be useful (usually **not**).

$|$  $-$  $| > Th$

Slide credit: Birgi Tamersoy

Frame Differencing

$Th = 25$



$Th = 50$

$Th = 100$

$Th = 200$

Mean Filter

- In this case the background is the mean of the previous n frames:

$$B(x,y,t) = \frac{1}{n}\sum_{i=0}^{n-1} I(x,y,t-i)$$

$$\Downarrow$$

$$|I(x,y,t) - \frac{1}{n}\sum_{i=0}^{n-1} I(x,y,t-i)| > Th$$

- For $n = 10$:

Estimated Background

Foreground Mask

Median Filter

- Assuming that the background is more likely to appear in a scene, we can use the median of the previous n frames as the background model:

$$B(x, y, t) = median\{I(x, y, t - i)\}$$
$$\Downarrow$$
$$|I(x, y, t) - median\{I(x, y, t - i)\}| > Th \text{ where } i \in \{0, \ldots, n - 1\}.$$

- For $n = 10$:

Estimated Background

Foreground Mask

Eliminates Moving Objects: Since moving objects appear in different locations across frames, they do not persist in the median computation, effectively removing them from the estimated background.

Average/Median Image

Alyosha Efros, CMU

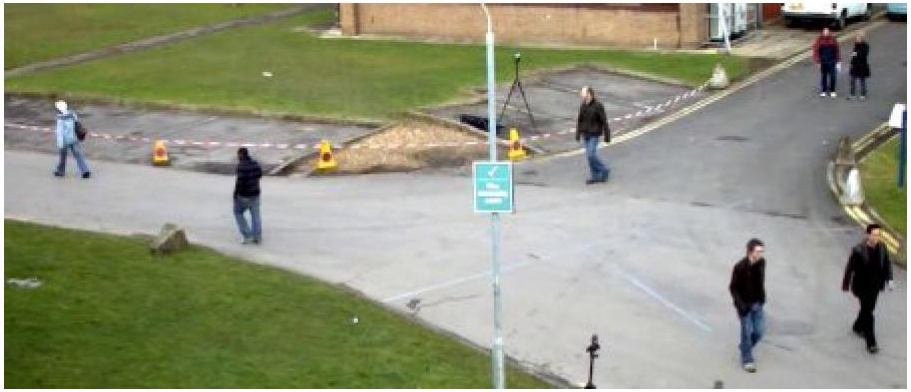
Background Subtraction

-

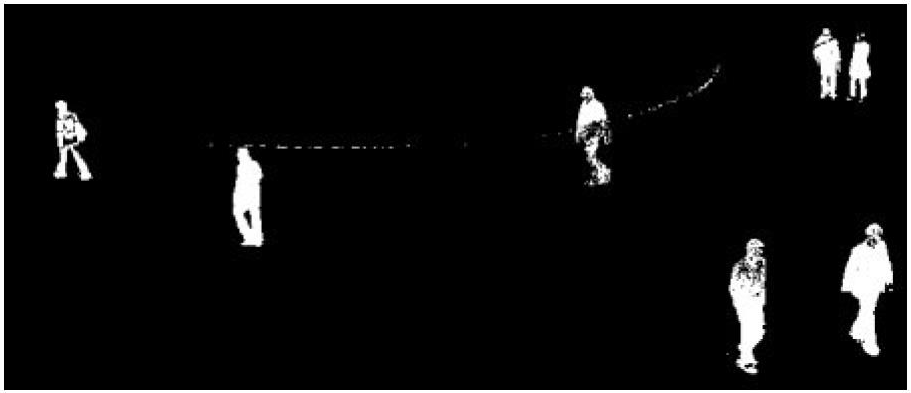
=

Alyosha Efros, CMU

Background Subtraction



Background Subtraction



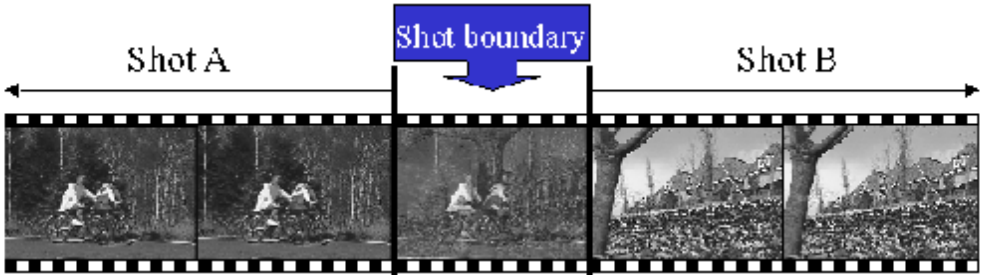
python background_subtraction.py --input vtest.avi

Motion Applications

- Background subtraction (segmentation)
- Shot boundary detection
  - Commercial video is usually composed of *shots* or sequences showing the same objects or scene
  - Goal: segment video into shots for summarization and browsing (each shot can be represented by a single keyframe in a user interface)
  - Difference from background subtraction: the camera is not necessarily stationary

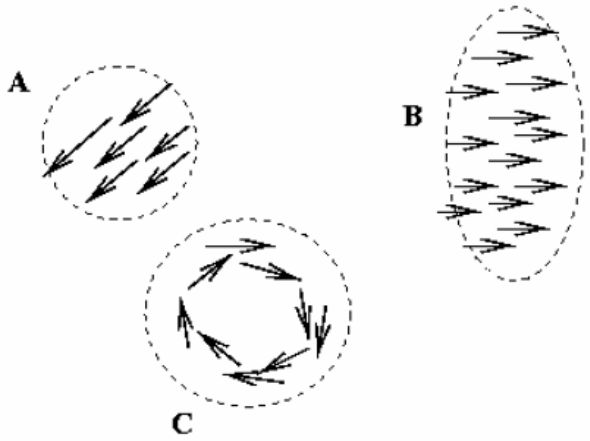
Motion Applications

- Background subtraction (segmentation)
- Shot boundary detection
  - For each frame
    - Compute the distance between the current frame and the previous one
      - Pixel-by-pixel differences
      - Differences of color histograms
      - Block comparison
    - If the distance is greater than some threshold, classify the frame as a shot boundary



Motion Applications

- Background subtraction (segmentation)
- Shot boundary detection
- Motion segmentation
  - Segment the video into multiple coherently moving objects



Motion Applications

- Background subtraction (segmentation)
- Shot boundary detection
- Motion segmentation
  - Segment the video into multiple coherently moving objects

Applications of segmentation to video

- Background subtraction
  - Form an initial background estimate
  - For each frame:
    - Update estimate using a moving average
    - Subtract the background estimate from the frame
    - Label as foreground each pixel where the magnitude of the difference is greater than some threshold
    - Use median filtering to “clean up” the results

Video Object Segmentation

Generate pixel level foreground masks for an object(s) across the frames of a video

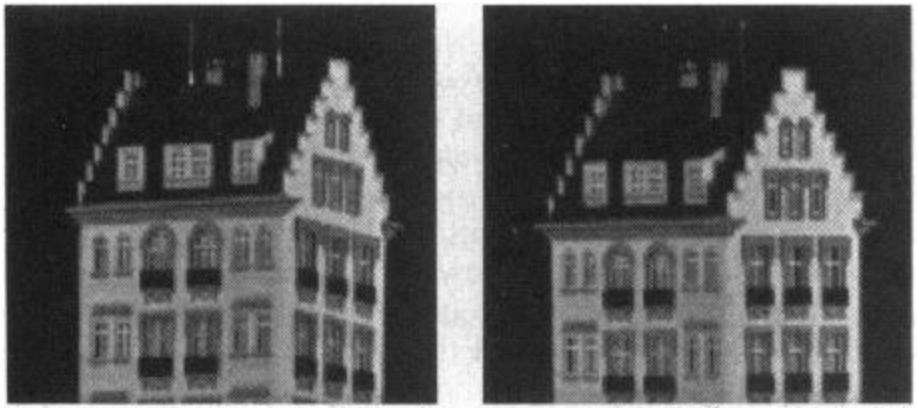
Click Carving Results

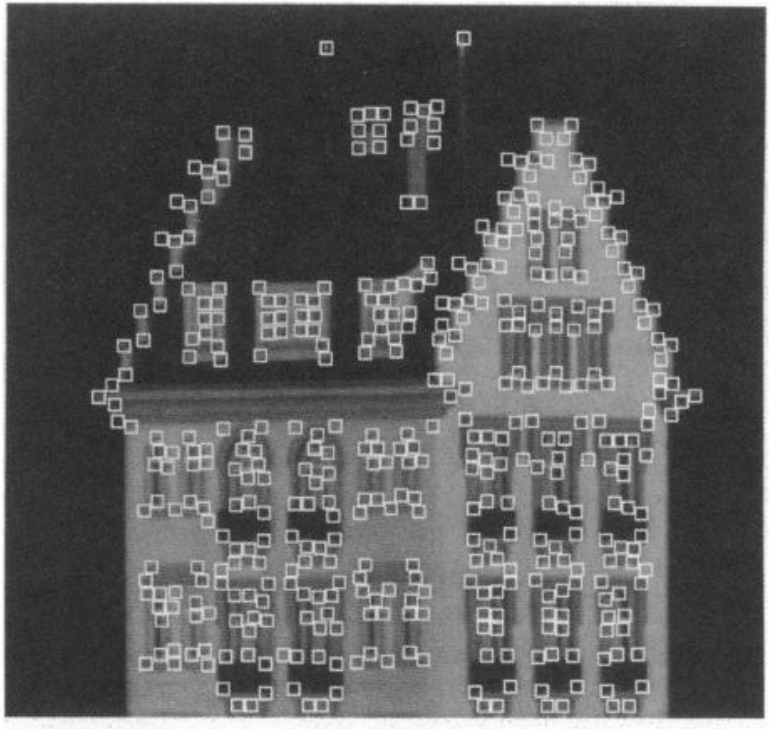
Using only 1-2 clicks

Slide credit: Suyog Jain

Feature tracking

- Many problems, such as structure from motion require matching points
- If motion is small, tracking is an easy way to get them

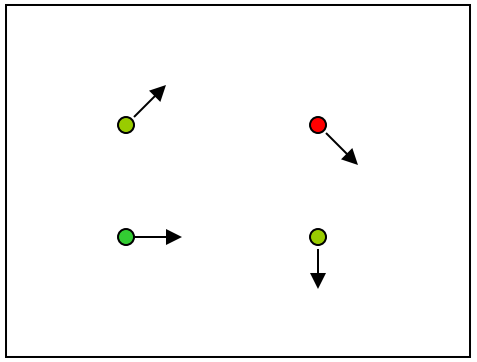




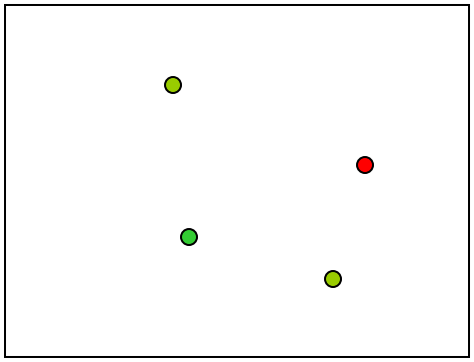
Feature tracking - Challenges

- Figure out which features can be tracked
- Efficiently track across frames
- Some points may change appearance over time (e.g., due to rotation, moving into shadows, etc.)
- Drift: small errors can accumulate as appearance model is updated
- Points may appear or disappear: need to be able to add/delete tracked points

Feature tracking



$I(x,y,t)$



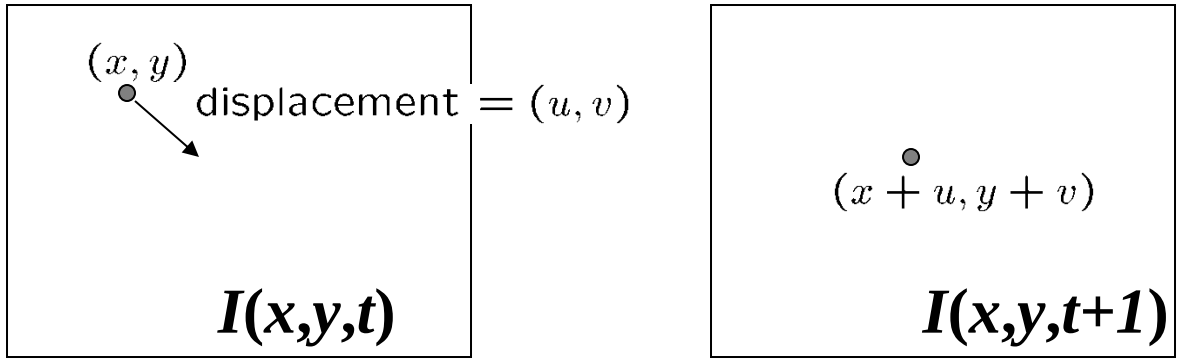
$I(x,y,t+1)$

- Given two subsequent frames, estimate the point translatio
- Solution: Lucas-Kanade Tracker

Key assumptions of Lucas-Kanade Tracker

- Brightness constancy:
  - The intensity of a pixel does not change significantly between two consecutive frames.
- Small motion assumption:
  - The displacement of a pixel between two frames is small.
- Spatial coherence:
  - Neighboring pixels have similar motion.

The brightness constancy constraint



- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ **at** (x, y, t) **to linearize the right side:**

Image derivative along x **Difference over frames**

$$I(x+u, y+v, t+1) \approx I(x, y, t) + I_x \cdot u + I_y \cdot v + I_t$$

$$I(x+u, y+v, t+1) - I(x, y, t) = I_x \cdot u + I_y \cdot v + I_t$$

So: $$I_x \cdot u + I_y \cdot v + I_t \approx 0 \quad \rightarrow \nabla \mathrm{I} \cdot \left[\mathrm{u}\ \mathrm{v}\right]^{\mathrm{T}} + \mathrm{I}_{\mathrm{t}} = 0$$

The Brightness Constancy Constraint

Can we use this equation to recover image motion (u,v) at each pixel?

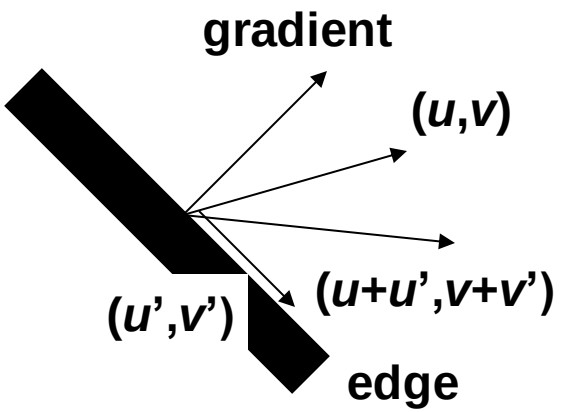
$$\nabla I \cdot \begin{bmatrix} u & v \end{bmatrix}^T + I_t = 0$$

- How many equations and unknowns per pixel?
- One equation (this is a scalar equation!), two unknowns (u,v)

The component of the motion perpendicular to the gradient (i.e., parallel to the edge) cannot be measured

If (*u*, *v*) satisfies the equation, so does (*u+u'*, *v+v'*) if

$$\nabla I \cdot \begin{bmatrix} u' & v' \end{bmatrix}^T = 0$$



Ambiguity in Optical Flow Along Edges

$$\nabla I \cdot [u \quad v]^T + I_t = 0$$

If a motion vector (u, v) satisfies this equation, then any motion vector of the form:

$$(u + u', v + v')$$

also satisfies the equation if:

$$\nabla I \cdot [u' \quad v']^T = 0$$

This means that we can add **any motion component** (u', v') **that is perpendicular to the image gradient** ∇I without violating the constraint.

Why Does This Happen?

The **Optical Flow Constraint Equation** only tells us about motion **along the gradient direction** but does **not** provide information about motion **perpendicular to it**. This results in **infinite possible solutions along the edge direction**, causing what is known as the **Aperture Problem**.

1. Interpreting $\nabla I \cdot [u' \quad v']^T = 0$

- The dot product $\nabla I \cdot [u' \quad v']^T = 0$ implies that (u', v') is **orthogonal** (perpendicular) to the gradient.
- This means we can **move freely along the edge direction** without changing image intensity.

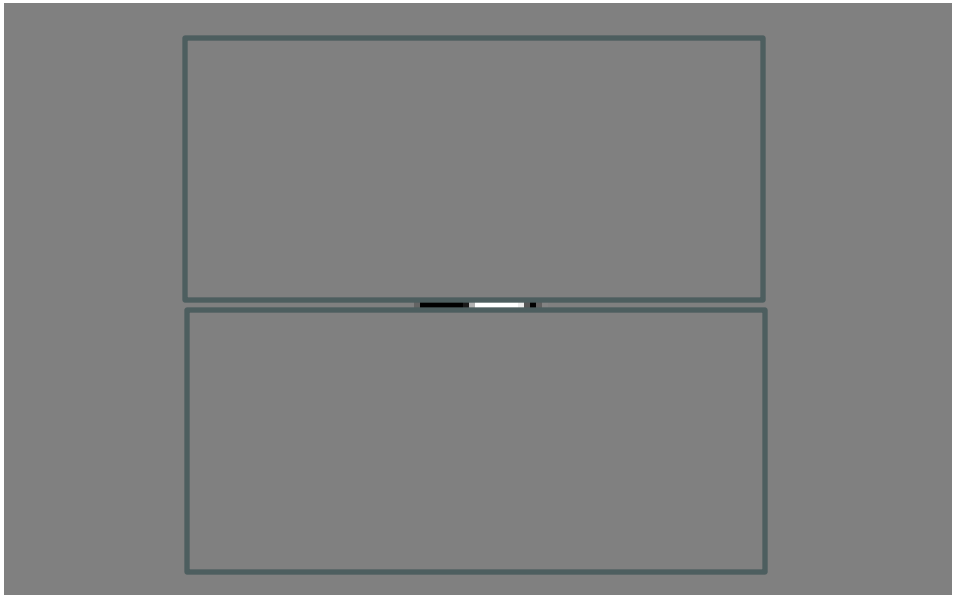
2. Intuition: What Happens Along an Edge?

- Suppose you have an **edge in an image** (e.g., a vertical stripe).
- The intensity changes **perpendicular to the edge** (along the gradient direction).
- However, moving **along the edge does not change the intensity**, making motion **ambiguous**.

In such cases:

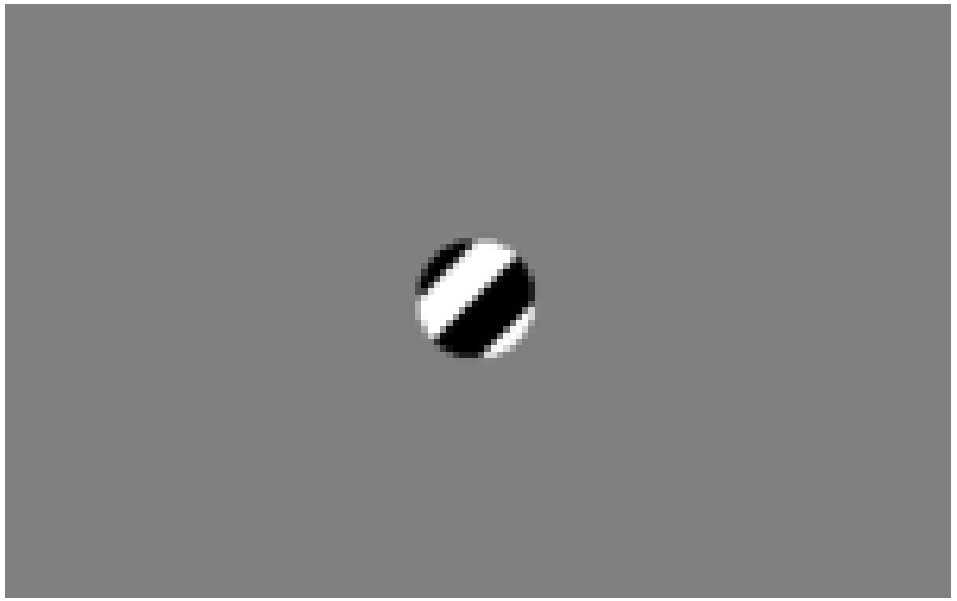
- The optical flow constraint can only determine **motion perpendicular to the edge**.
- Motion **parallel to the edge remains unknown**, leading to **infinite possible solutions** in that direction.

The barber pole illusion



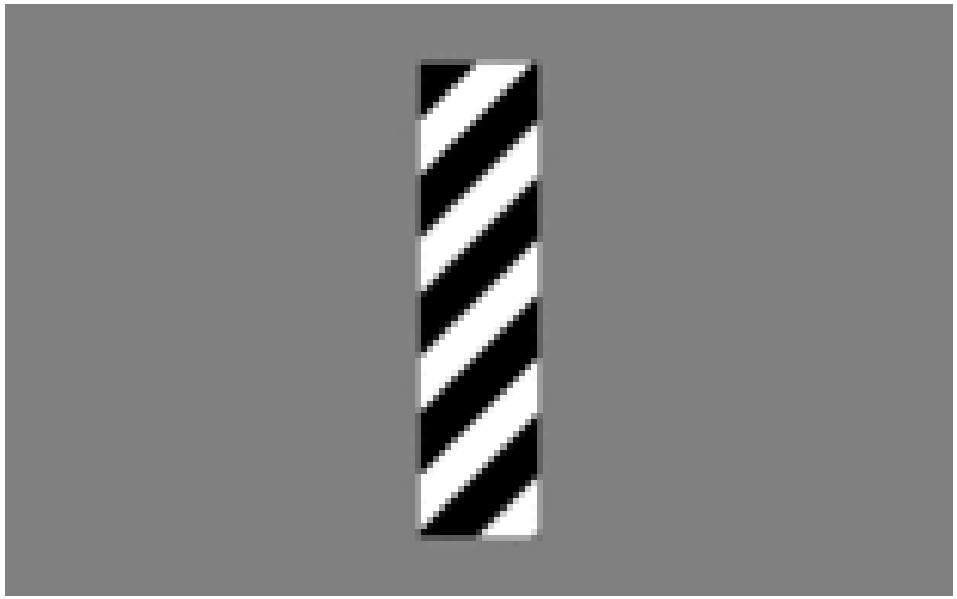
http://en.wikipedia.org/wiki/Barberpole_illusion

The barber pole illusion



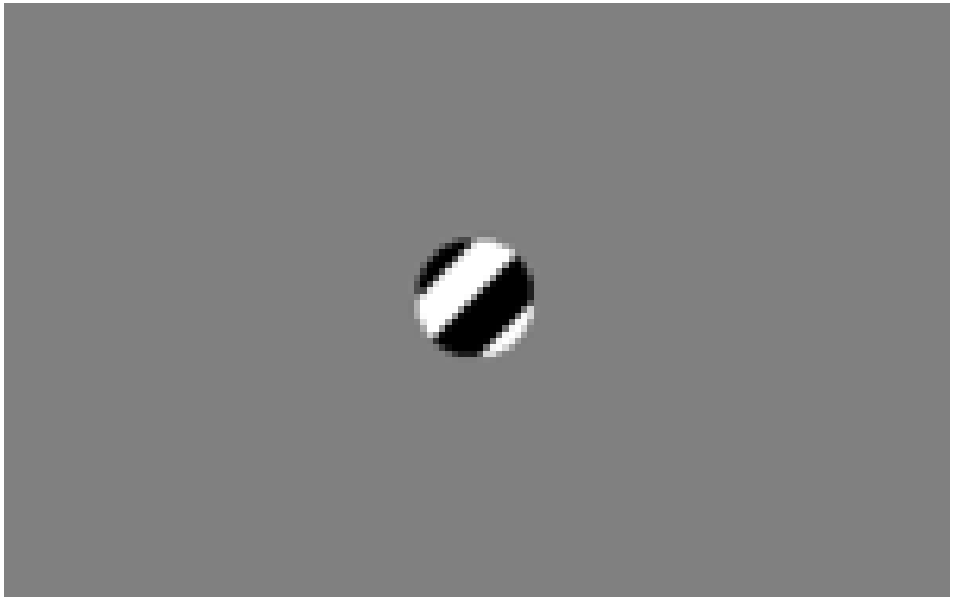
http://en.wikipedia.org/wiki/Barberpole_illusion

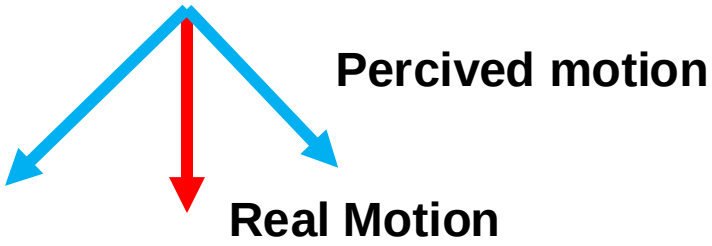
The barber pole illusion



http://en.wikipedia.org/wiki/Barberpole_illusion

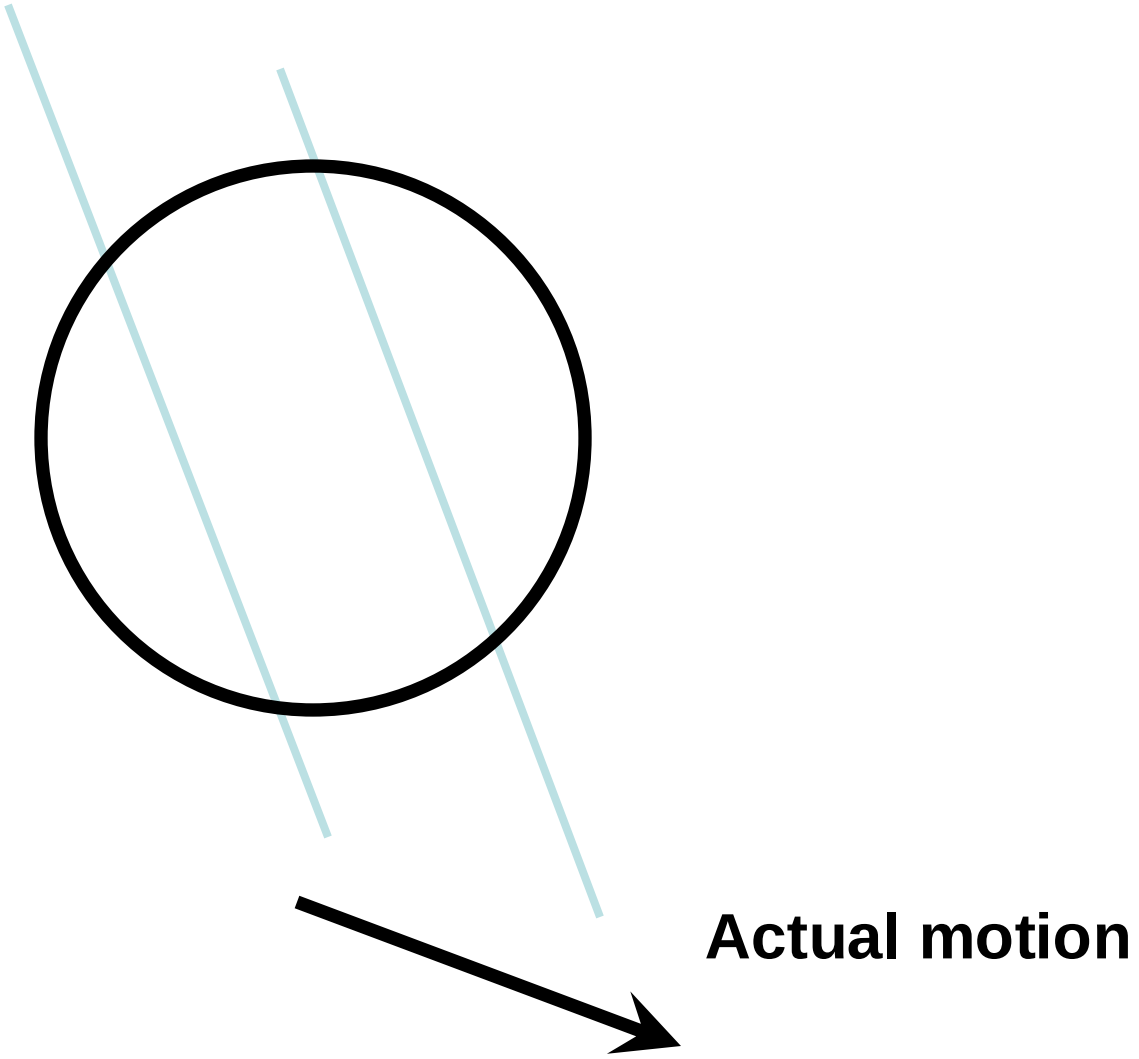
The barber pole illusion



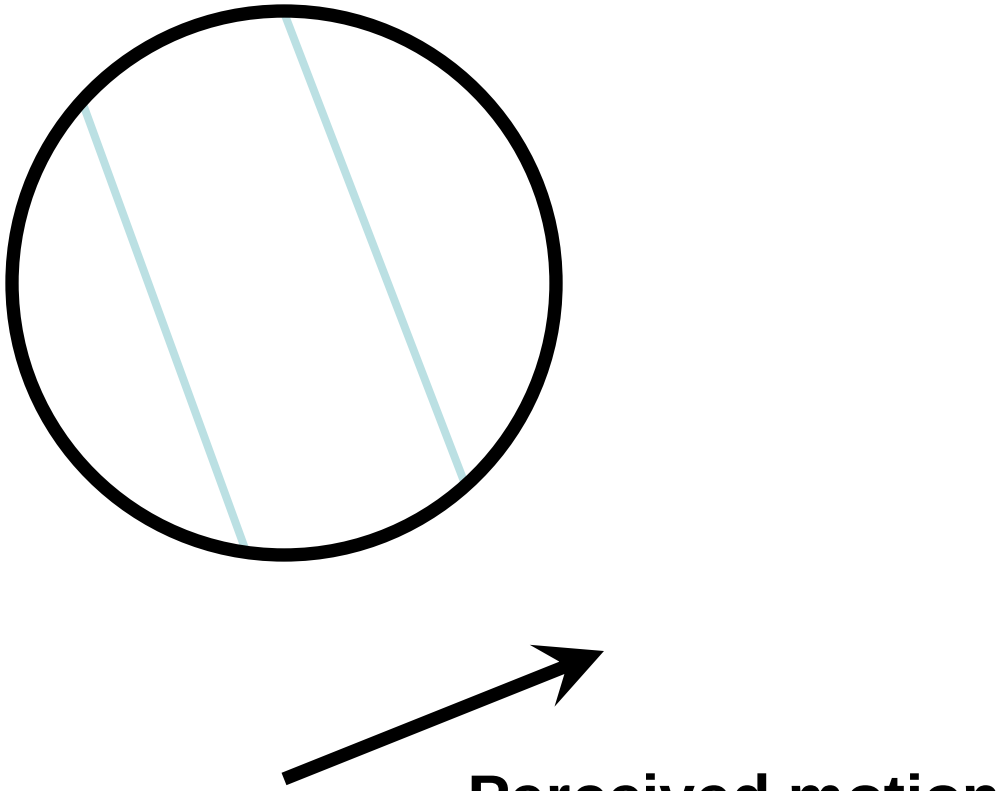


http://en.wikipedia.org/wiki/Barberpole_illusion

The aperture problem



The aperture problem



The **Aperture Problem**

- The **Aperture Problem** occurs when **motion is ambiguous** due to **limited information from a small viewing window**.
- This happens because **motion along an edge can be detected, but motion along the edge itself remains unknown**.
- When observing motion through a small aperture (window), only the component of motion **perpendicular to the edge** can be determined. The motion along the edge direction is **invisible**, leading to multiple

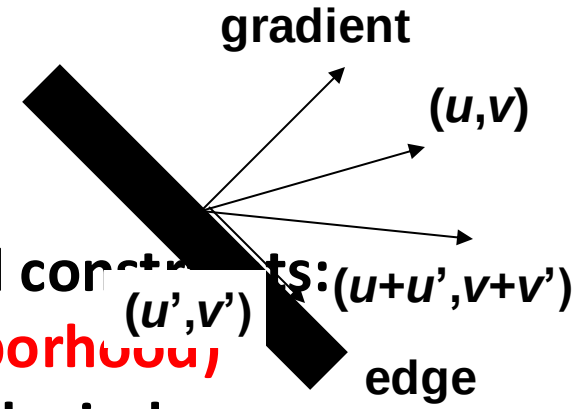
How to Fix the Aperture Problem?

- The **Optical Flow Constraint Equation** is:

$$I_x u + I_y v + I_t = 0$$

Since we have **one equation with two unknowns**, the motion component **parallel to the edge remains ambiguous**.

If $\nabla I \cdot [u' \quad v']^T = 0$, then we can add **any motion along the edge direction**, leading to **infinite solutions**.



To fully determine motion, we need additional constraints:

Solution : Lucas-Kanade Method (Local Neighborhood)

- **Assumes motion is constant within a small window around each pixel.**
- **Uses multiple pixels (instead of a single pixel) to construct an overdetermined system.**
- **Solves for motion using Least Squares estimation.**

Solving the ambiguity…

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- How to get more equations for a pixel?
- **Spatial coherence constraint**
  - Assume the pixel's neighbors have the same (u,v)
  - If we use a 5x5 window, that gives us 25 equations per pixel

$$0 = I_t(\mathbf{p_i}) + \nabla I(\mathbf{p_i}) \cdot [u\ v]$$

$$\begin{bmatrix} I_x(\mathbf{p_1}) & I_y(\mathbf{p_1}) \\ I_x(\mathbf{p_2}) & I_y(\mathbf{p_2}) \\ \vdots & \vdots \\ I_x(\mathbf{p_{25}}) & I_y(\mathbf{p_{25}}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p_1}) \\ I_t(\mathbf{p_2}) \\ \vdots \\ I_t(\mathbf{p_{25}}) \end{bmatrix}$$

Solving the ambiguity…

- Least squares problem:

$$\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix}$$

$$\underset{25\text{x}2}{A} \quad \underset{2\text{x}1}{d} = \underset{25\text{x}1}{b}$$

Matching patches across images

- Overconstrained linear system

$$\begin{bmatrix} I_x(\mathrm{p}_1) & I_y(\mathrm{p}_1) \\ I_x(\mathrm{p}_2) & I_y(\mathrm{p}_2) \\ \vdots & \vdots \\ I_x(\mathrm{p}_{25}) & I_y(\mathrm{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathrm{p}_1) \\ I_t(\mathrm{p}_2) \\ \vdots \\ I_t(\mathrm{p}_{25}) \end{bmatrix} \qquad \underset{25\text{x}2}{A} \quad \underset{2\text{x}1}{d} = \underset{25\text{x}1}{b}$$

Least squares solution for *d* given by $(A^T A)\ d = A^T b$

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $\qquad\qquad$ $A^T b$

The summations are over all pixels in the K x K window

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $\qquad$ $A^T b$

When is this solvable? I.e., what are good points to track?

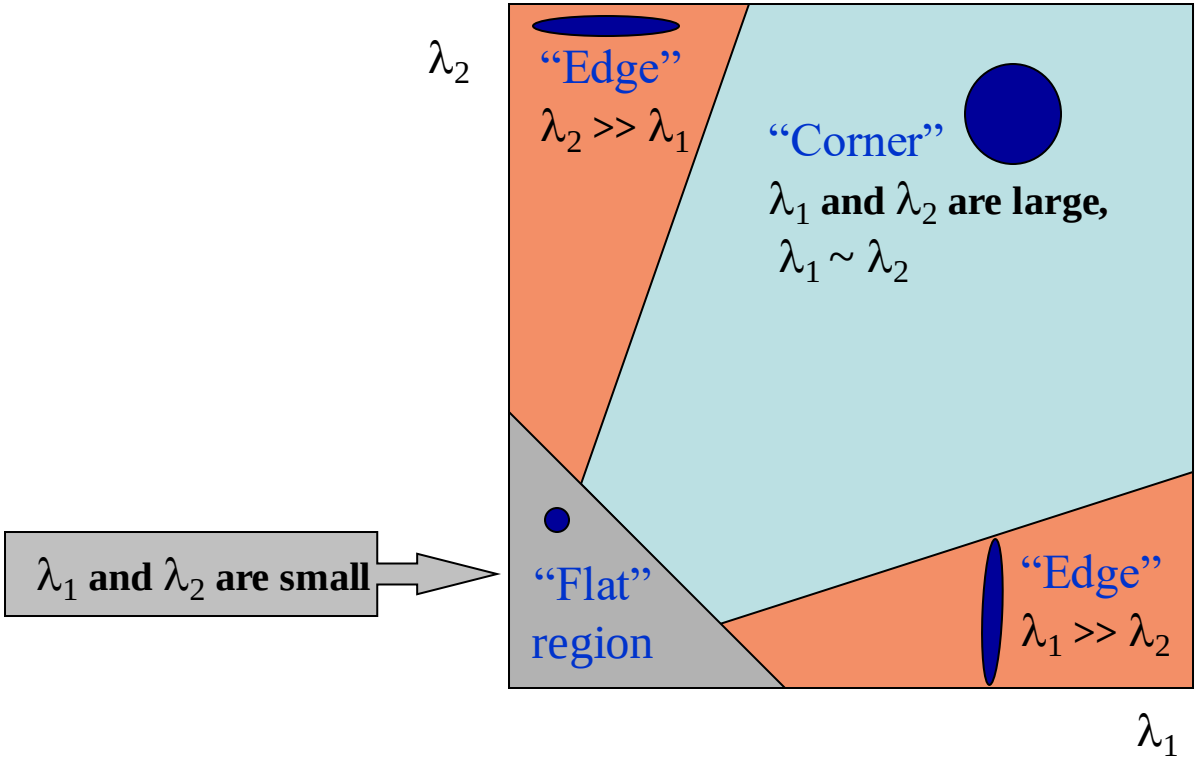
- **A^TA should be invertible**
- **A^TA should not be too small due to noise**
  - **eigenvalues λ_1 and λ_2 of A^TA should not be too small**
- **A^TA should be well-conditioned**
  - **λ_1 / λ_2 should not be too large (λ_1 = larger eigenvalue)**

Does this remind you of anything?

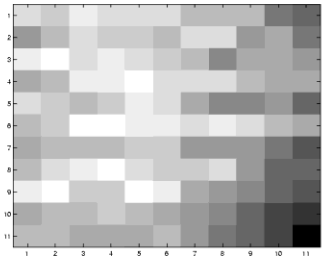
Criteria for Harris corner detector

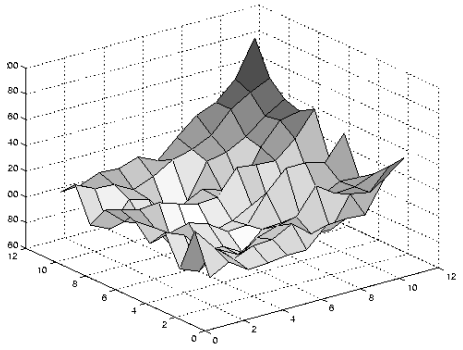
Recall: second moment matrix

Classification of windows using eigenvalues of the second moment matrix:



Low texture region

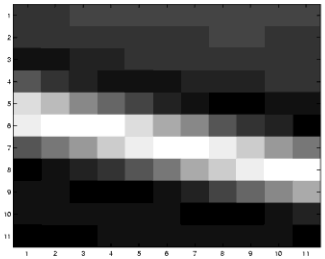


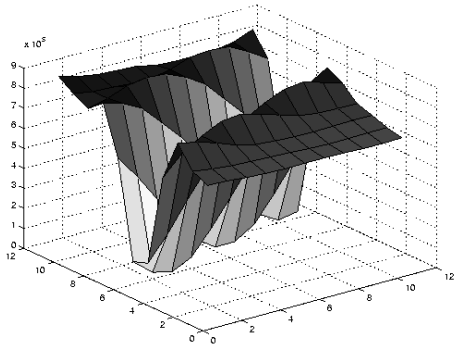


$$\sum \nabla I (\nabla I)^T$$

- **gradients have small magnitude**
- **small λ_1, small λ_2**

Edge

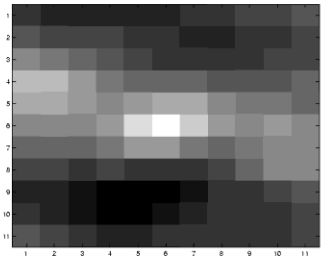


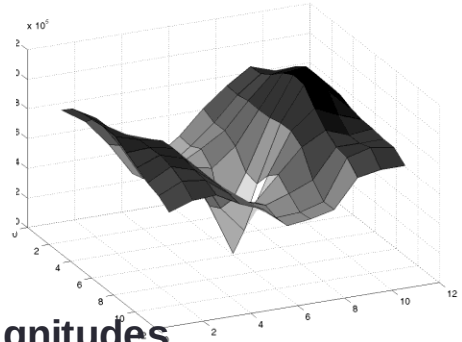


$$\sum \nabla I (\nabla I)^T$$

- large gradients, all the same
- large λ_1, small λ_2

High textured region

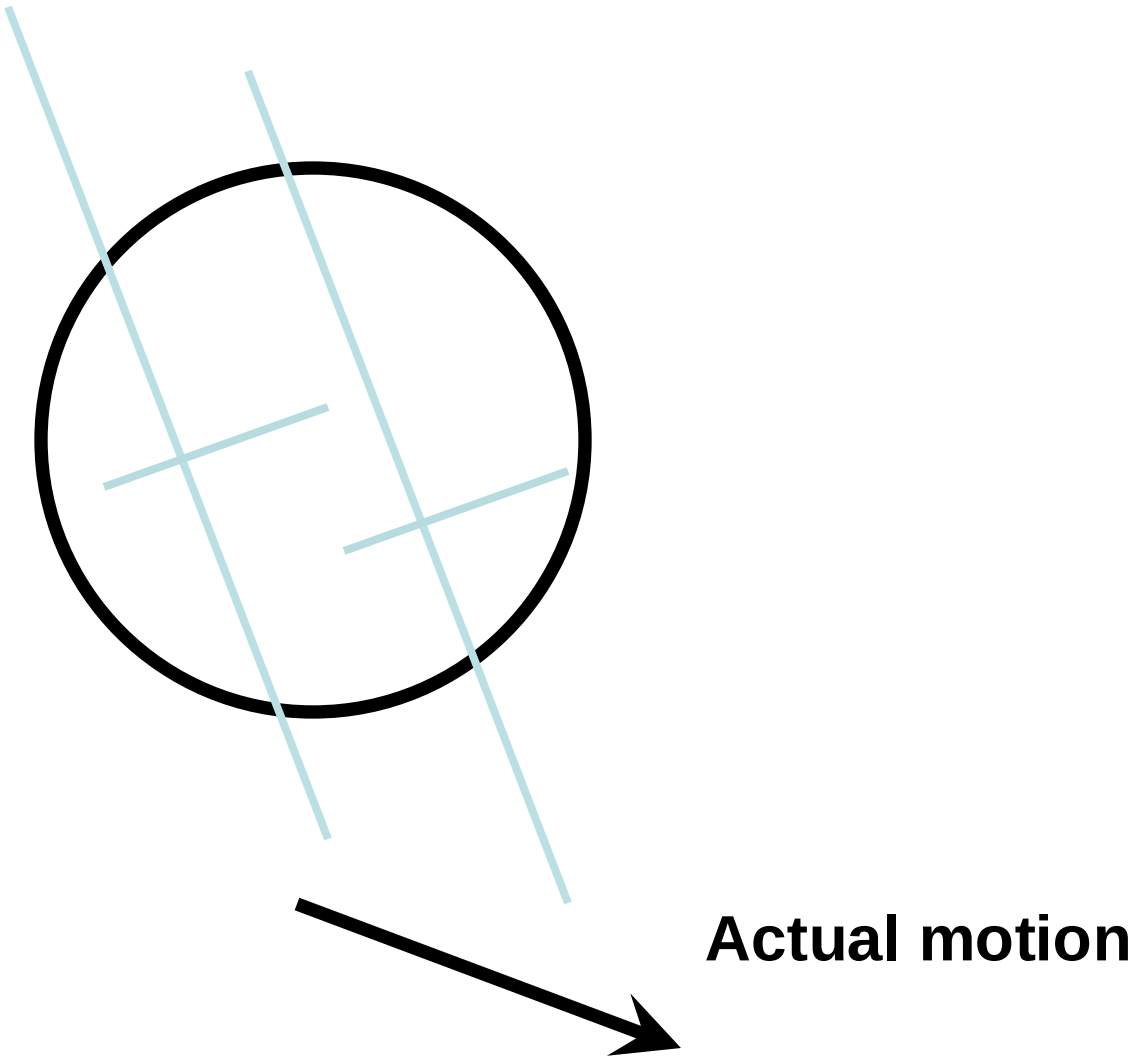




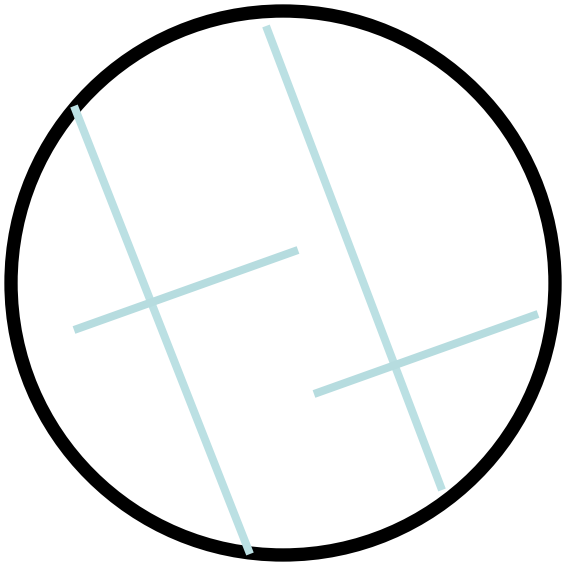
$$\sum \nabla I (\nabla I)^T$$

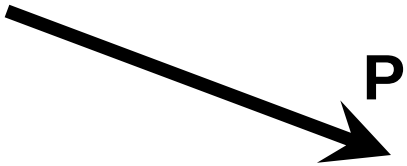
- **gradients are different, large magnitudes**
- **large λ_1, large λ_2**

The aperture problem resolved



The aperture problem resolved





Perceived motion

Shi-Tomasi feature tracker

- Find good features using eigenvalues of second-moment matrix (e.g., Harris detector or threshold on the smallest eigenvalue)
  - Key idea: "good" features to track are the ones whose motion can be estimated reliably

- Track from frame to frame with Lucas-Kanade
  - This amounts to assuming a translation model for frame-to-frame feature movement

- Check consistency of tracks by *affine* registration to the first observed instance of the feature
  - Affine model is more accurate for larger displacements
  - Comparing to the first frame helps to minimize drift

J. Shi and C. Tomasi. Good Features to Track. CVPR 1994.

Tracking example

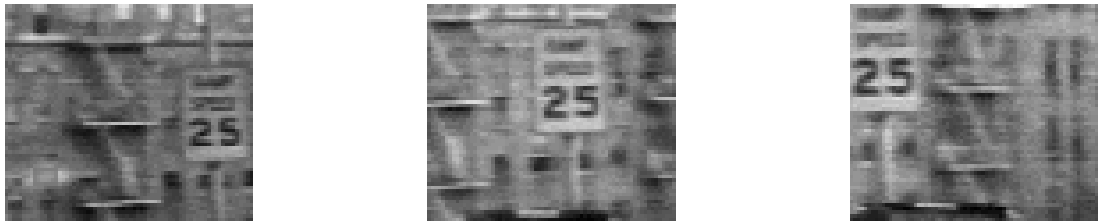


Figure 1: Three frame details from Woody Allen's *Manhattan*. The details are from the 1st, 11th, and 21st frames of a subsequence from the movie.

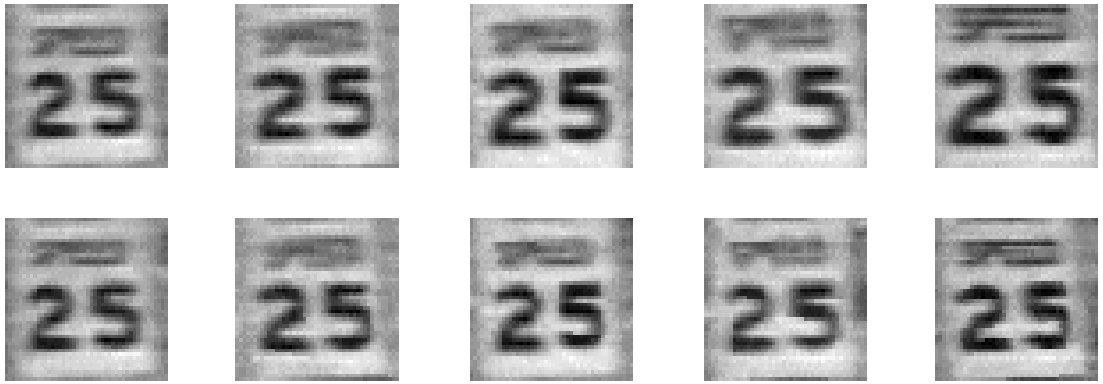


Figure 2: The traffic sign windows from frames 1,6,11,16,21 as tracked (top), and warped by the computed deformation matrices (bottom).

J. Shi and C. Tomasi. Good Features to Track. CVPR 1994.

Summary of KLT tracking

- Find a good point to track (harris corner)
- Use intensity second moment matrix and difference across frames to find displacement
- Iterate and use coarse-to-fine search to deal with larger movements
- When creating long tracks, check appearance of registered patch against appearance of initial patch to find points that have drifted

Implementation issues

- Window size
  - Small window more sensitive to noise and may miss larger motions (without pyramid)
  - Large window more likely to cross an occlusion boundary (and it's slower)
  - 15x15 to 31x31 seems typical

- Weighting the window
  - Common to apply weights so that center matters more (e.g., with Gaussian)

Why not just do local template matching?

- Slow (need to check more locations)
- Does not give subpixel alignment (or becomes much slower)
  - Even pixel alignment may not be good enough to prevent drift
- May be useful as a step in tracking if there are large movements

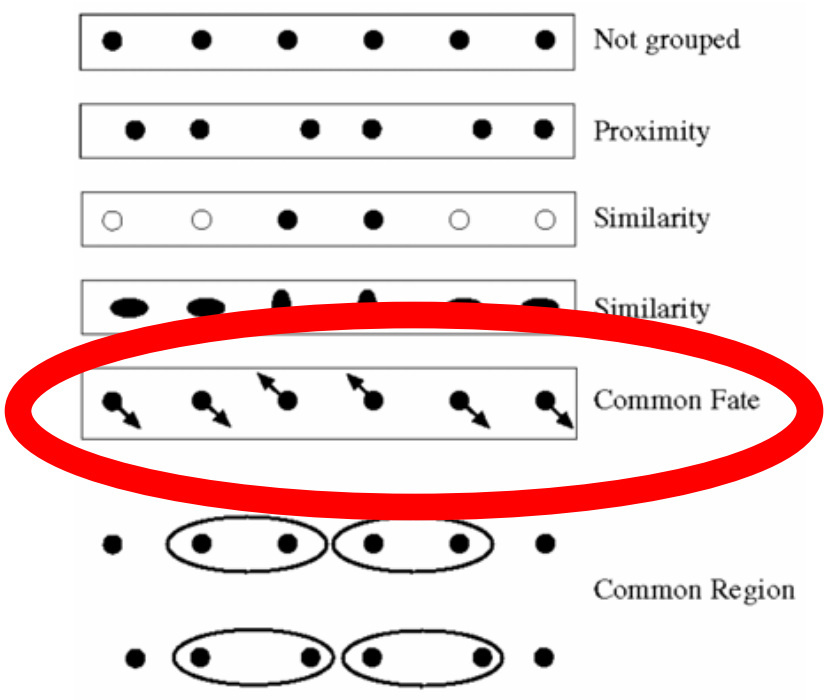
OPTICAL FLOW AND MOTION

Motion and perceptual organization

Gestalt psychology (Max Wertheimer, 1880-1943)

Motion and perceptual organization

- Sometimes, motion is the only cue



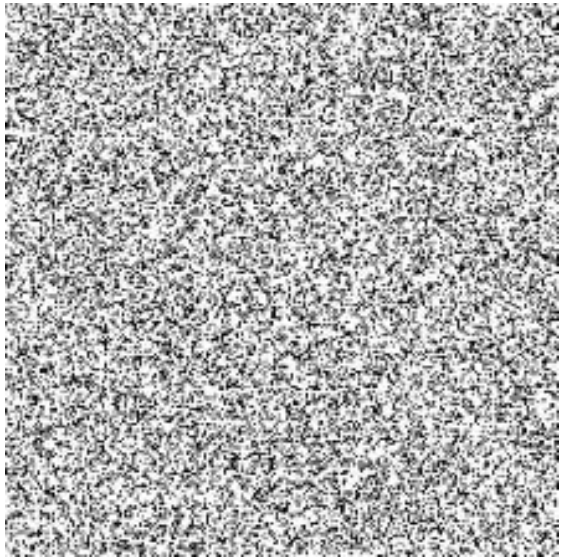
Gestalt psychology (Max Wertheimer, 1880-1943)

Motion and perceptual organization

- Sometimes, motion is the only cue

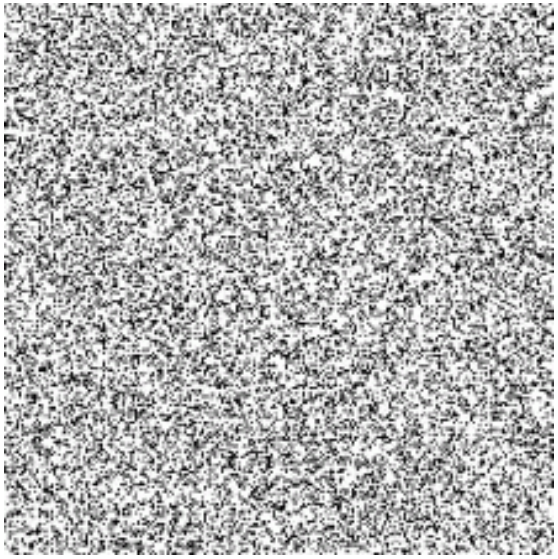
Motion and perceptual organization

- Sometimes, motion is the only cue



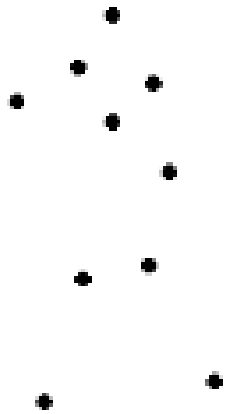
Motion and perceptual organization

- Sometimes, motion is the only cue



Motion and perceptual organization

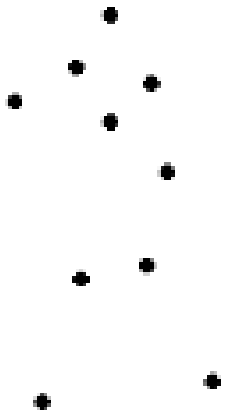
- Even “impoverished” motion data can evoke a strong percept



G. Johansson, “Visual Perception of Biological Motion and a Model For Its Analysis", *Perception and Psychophysics 14, 201-211, 1973.*

Motion and perceptual organization

- Even “impoverished” motion data can evoke a strong percept

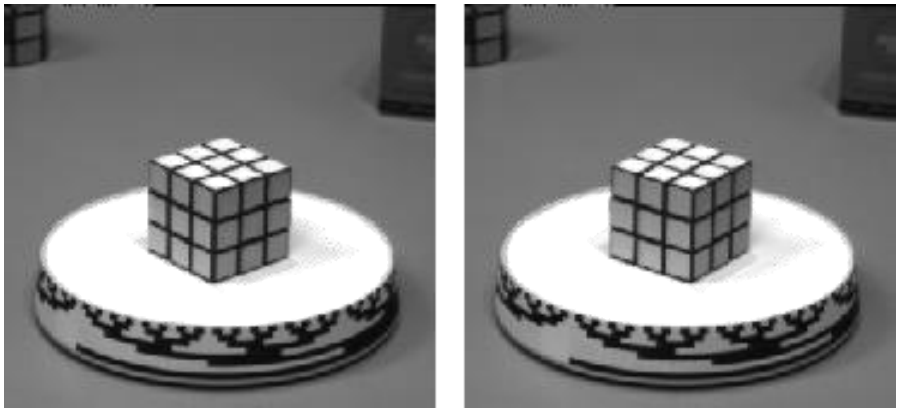


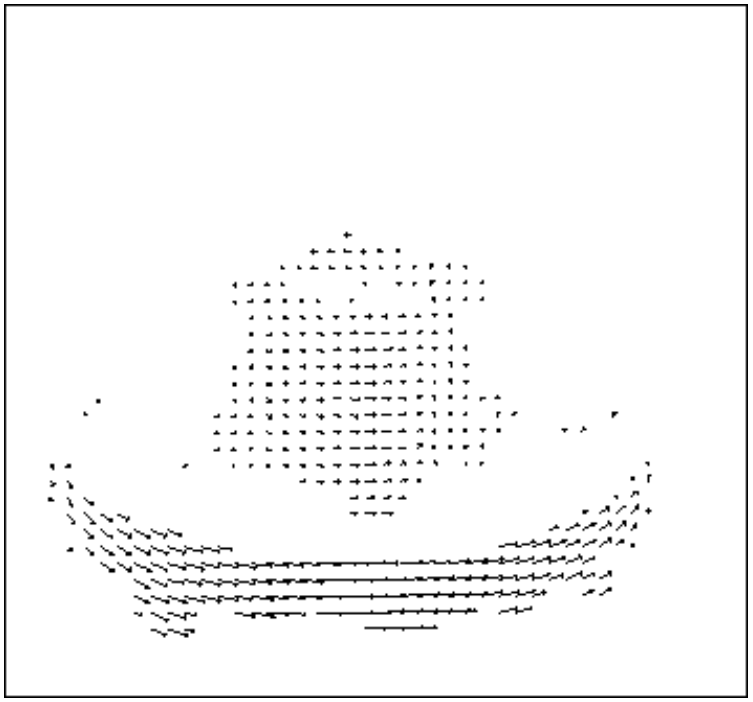
Uses of motion

- Estimating 3D structure
- Segmenting objects based on motion cues
- Learning and tracking dynamical models
- Recognizing events and activities
- Improving video quality (motion stabilization)

Motion field

- The motion field is the projection of the 3D scene motion into the image





What would the motion field of a non-rotating ball moving towards the camera look

Optical Flow

- Definition: the *apparent* motion of brightness patterns in the image
- Ideally, the same as the projected motion field
- Take care: apparent motion can be caused by lighting changes without any actual motion
  - Imagine a uniform rotating sphere under fixed lighting vs. a stationary sphere under moving illumination.

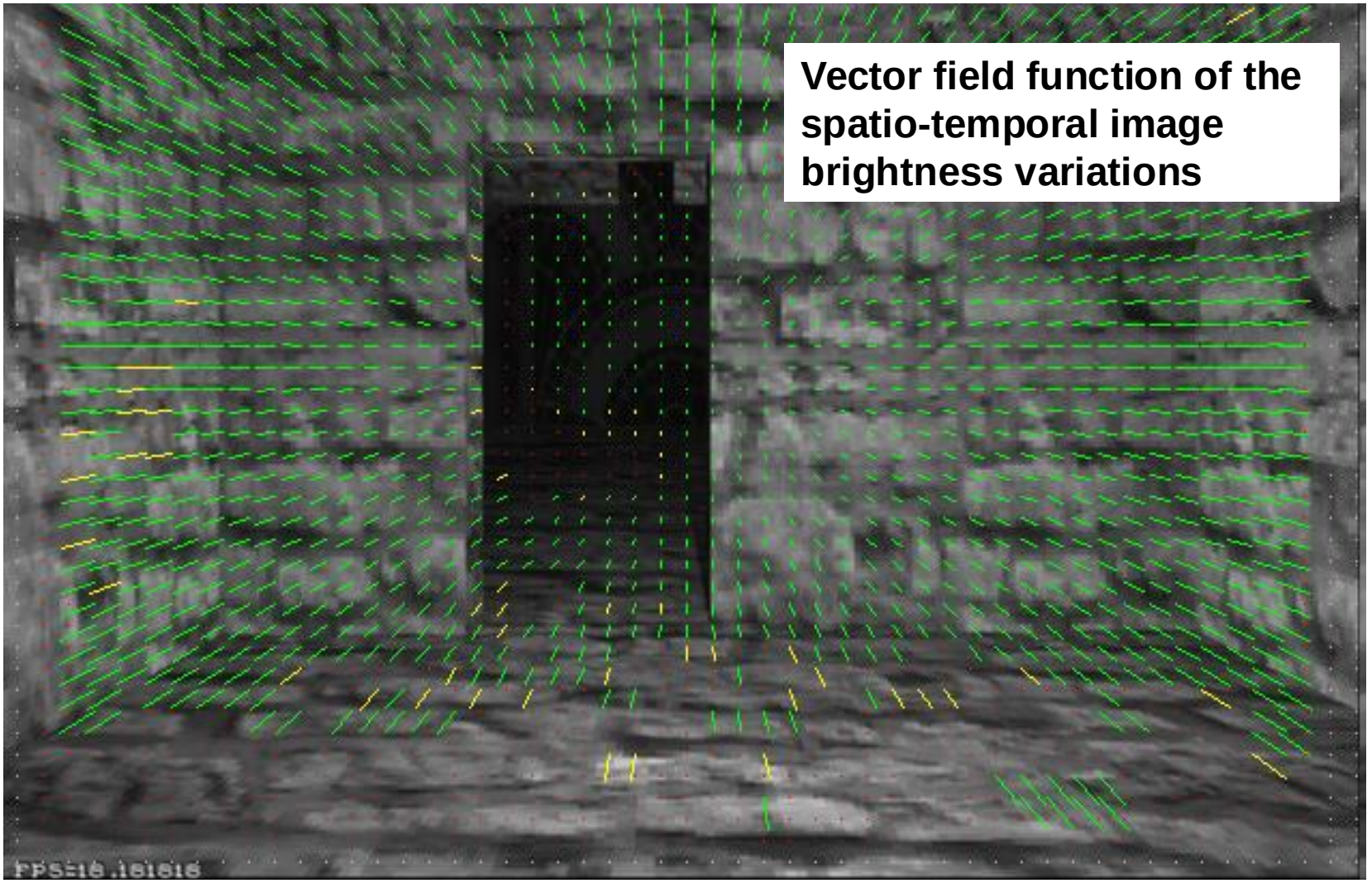
Lucas-Kanade Optical Flow

- Same as Lucas-Kanade feature tracking, but for each pixel
  - As we saw, works better for textured pixels
- Operations can be done one frame at a time, rather than pixel by pixel
  - Efficient

The Lucas-Kanade Algorithm

- https://youtu.be/D7r3-fHXvRU?t=1h40m54s

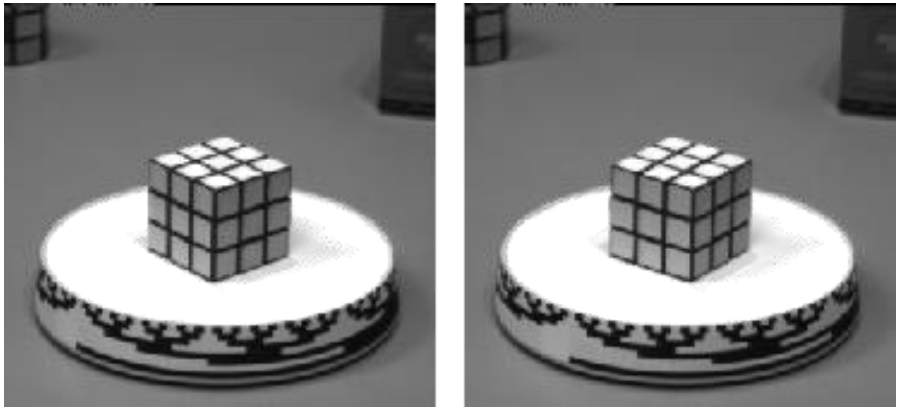
Optical flow

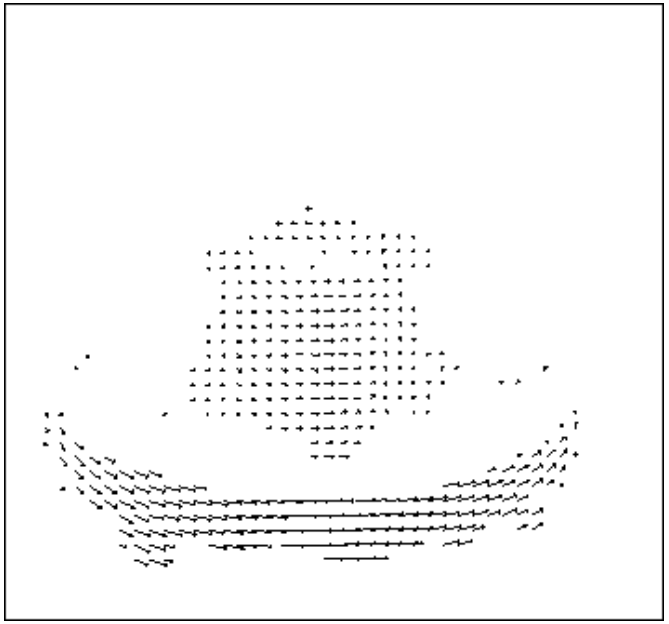


Picture courtesy of Selim Temizer - Learning and Intelligent Systems (LIS) Group, MIT

Motion estimation: Optical flow

Optic flow is the **apparent** motion of objects or surfaces





We started by estimating motion of a pixel, and now will consider motion of entire image

Thinks to Remember

- Are Lucas-Kanade Feature Tracking and Optical Flow Different?

- Lucas-Kanade **feature tracking** and **optical flow** are closely related but **not the same thing**. They use similar principles but serve **different purposes**.

Lucas-Kanade Feature Tracking

- Lucas-Kanade **feature tracking** (also known as **sparse optical flow**) is a method to track **key points** across frames rather than computing motion for all pixels.
- **How It Works**
  - First, it **selects good feature points** to track using methods like: **Shi-Tomasi corner detector** (goodFeaturesToTrack in OpenCV) or Harris corner detection.
  - Once key points are selected, it **applies Lucas-Kanade optical flow** only on those points.
  - The tracking happens over **multiple frames**, so

Lucas-Kanade Optical Flow

- Lucas-Kanade **optical flow** is a method to estimate the **motion of pixels** between two consecutive frames
- Assumes: **Small motion** and **Brightness constancy**.
- It computes **dense motion estimates**, meaning **every pixel** (or at least every pixel in a specific region) can have an estimated motion.
- **Characteristics**
  - Estimates motion for **all pixels in a region** (or even the whole image).

Errors in assumptions

- A point does not move like its neighbors
  - Motion segmentation
- Brightness constancy does not hold
  - Do exhaustive neighborhood search with normalized correlation - tracking features – maybe SIFT
- The motion is large (larger than a pixel)
  1. Not-linear: Iterative refinement
  2. Local minima: coarse-to-fine estimation

Solutions: Motion is Larger Than a Pixel

- **Solution 1: Iterative Refinement**
- Instead of computing u, v in one step, we refine it over **multiple iterations**:
  - Start with an **initial estimate** (e.g., assume small motion).
  - **Warp** the second frame using the estimate.
  - **Recalculate flow** using the residual error.
  - Repeat until **convergence**.
- This makes the **linear approximation more accurate** in each step.

Dealing with larger movements: Iterative Refinement

1. Initialize (x',y') = (x,y)
2. Compute (u,v) by

Original (x,y) position

$I_t = I(x', y', t+1) - I(x, y, t)$

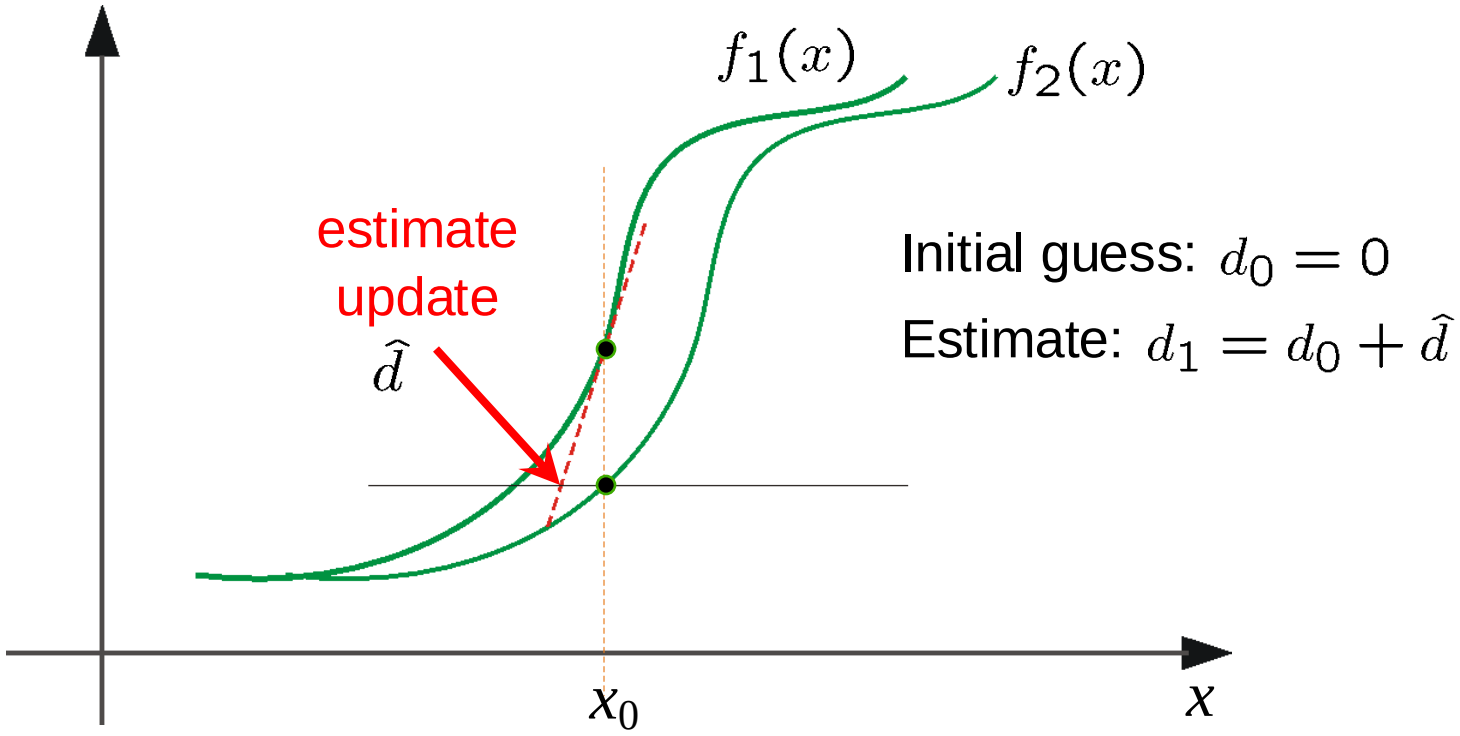
$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature patch in first image

displacement

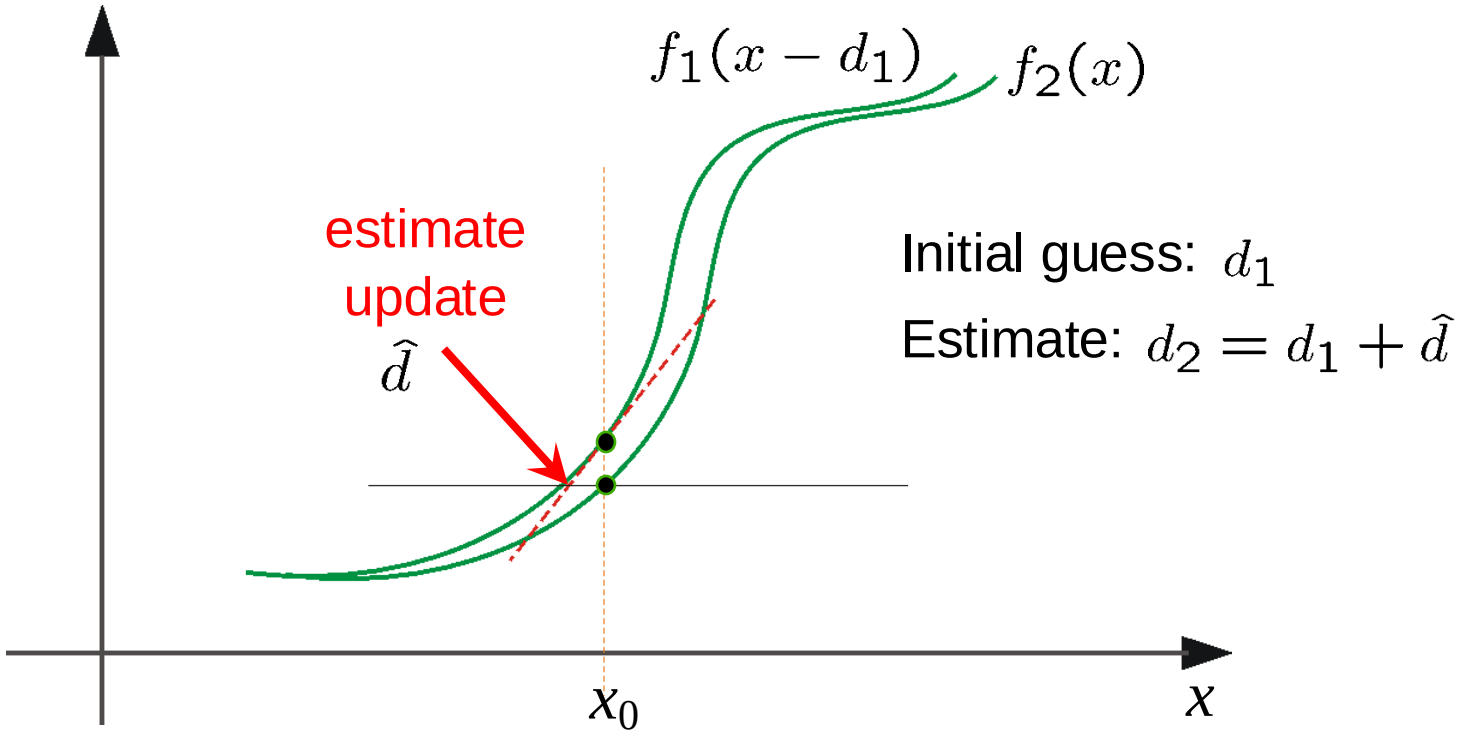
3. Shift window by (u, v): `x'=x'+u; y'=y'+v;`
4. Warp I_t towards I_{t+1} using the estimated flow field (u,v)
5. Repeat steps 2-4 until small change
   - Use interpolation for subpixel values

Optical Flow: Iterative Estimation

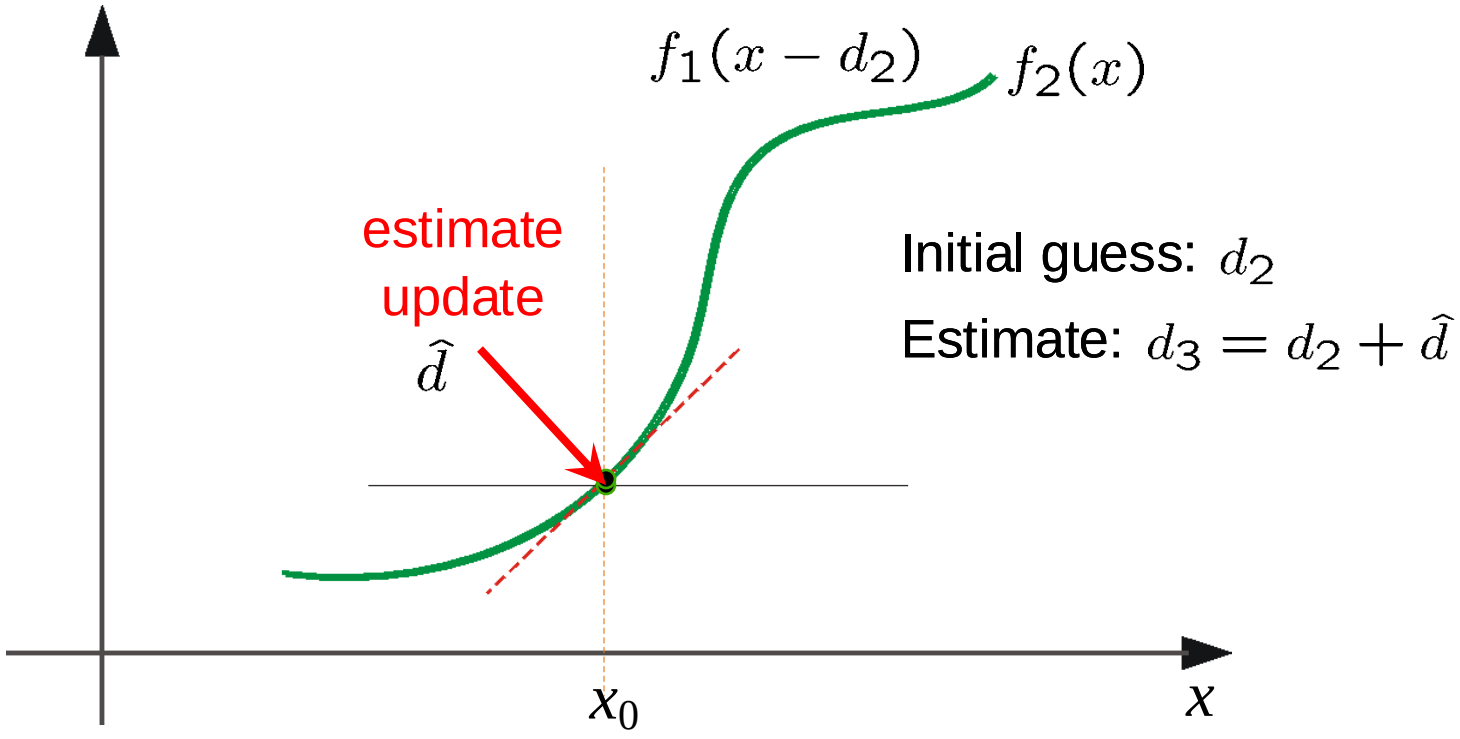


(using d for *displacement* here instead of u)

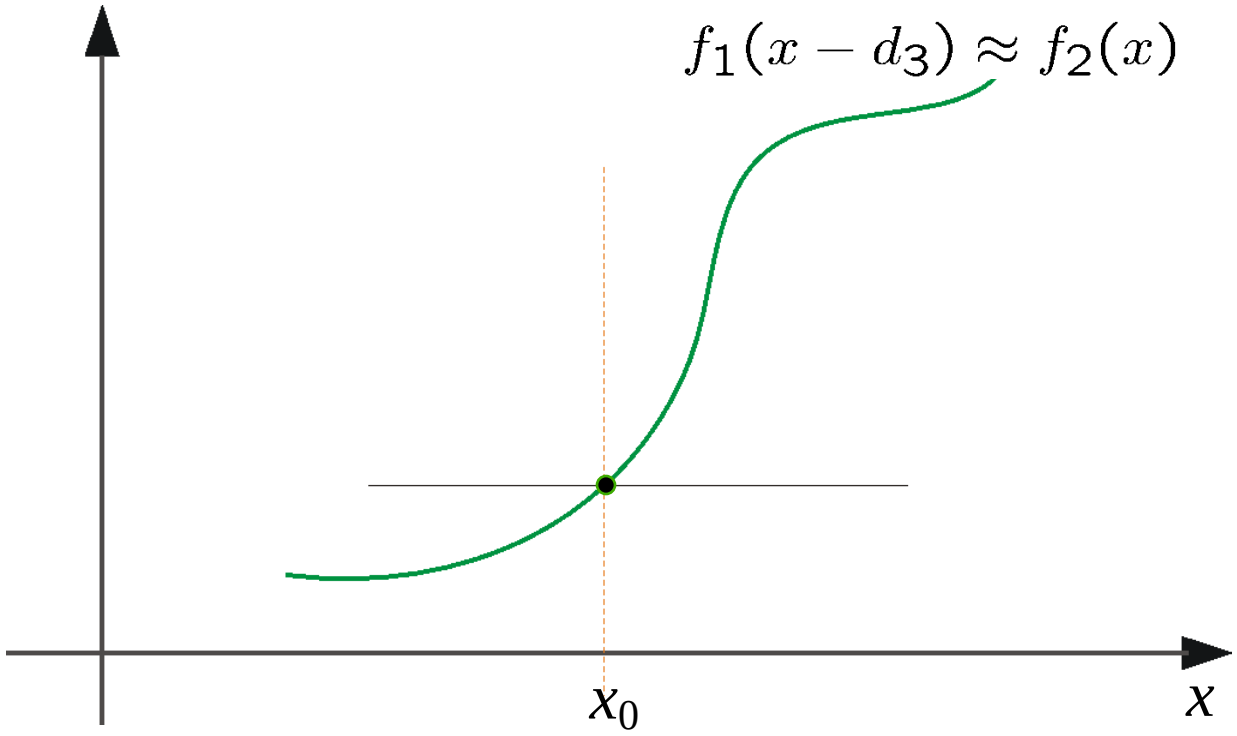
Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation



Optical Flow: Iterative Estimation

- Some Implementation Issues:
  - Warping is not easy (ensure that errors in warping are smaller than the estimate refinement)
  - Often useful to low-pass filter the images before motion estimation (for better derivative estimation, and linear approximations to image intensity)

Solutions: Motion is Larger Than a Pixel

- **Solution 2: Coarse-to-Fine (Pyramidal) Estimation**
- Instead of computing optical flow on the **full-resolution image**, use:
  - A **Gaussian Pyramid**: Downsample the image to create a **multi-resolution representation**.
  - Compute flow at the **lowest resolution (coarsest level)**.
  - **Upsample the flow** and refine it at each level until reaching full resolution.
- This allows the algorithm to **capture large displacements** at low resolution and refine

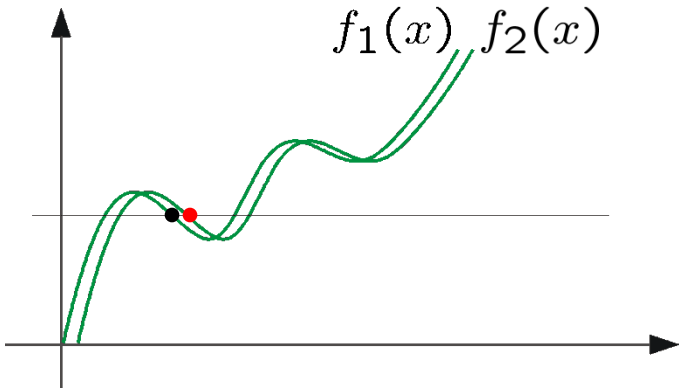
Revisiting the small motion assumption

- Is this motion small enough?
  - Probably not—it's much larger than one pixel
  - How might we solve this problem?

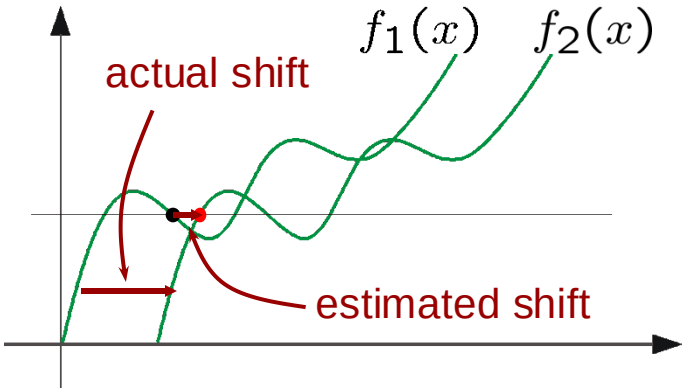
Optical Flow: Aliasing

Temporal aliasing causes ambiguities in optical flow because images can have many pixels with the same intensity.

I.e., how do we know which ‘correspondence’ is correct?



nearest match is correct (no aliasing)



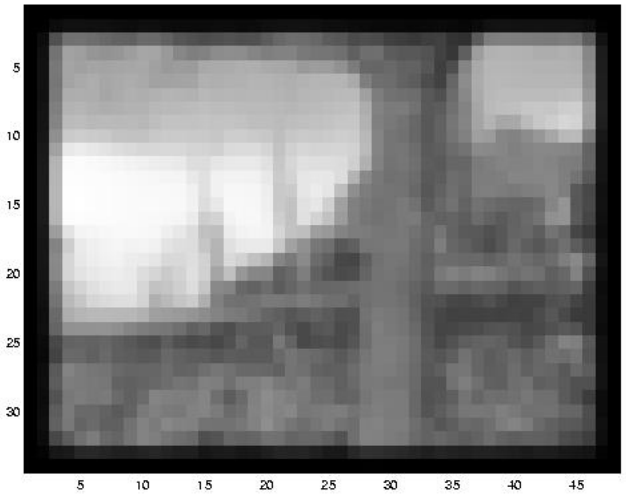
nearest match is incorrect (aliasing)

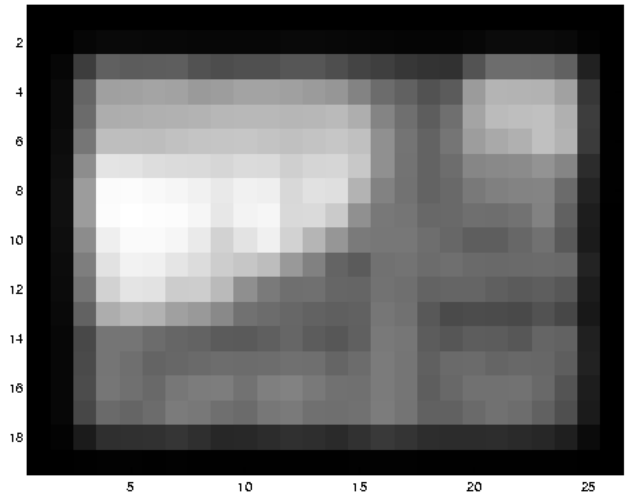
To overcome aliasing: coarse-to-fine estimation.

Solutions: Motion is Larger Than a Pixel

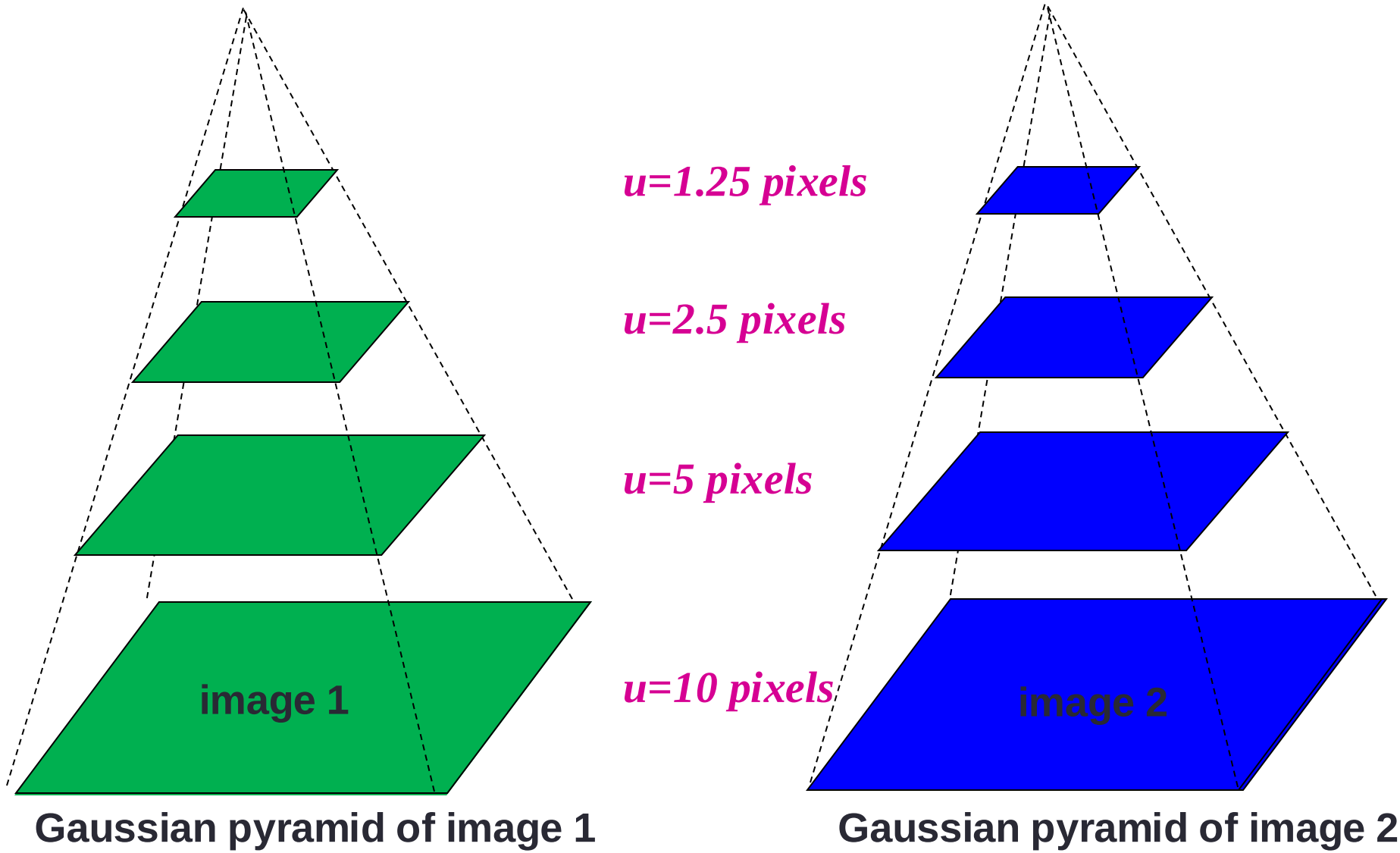
- **Solution 2: Coarse-to-Fine (Pyramidal) Estimation**
- Instead of computing optical flow on the **full-resolution image**, use:
  - A **Gaussian Pyramid**: Downsample the image to create a **multi-resolution representation**.
  - Compute flow at the **lowest resolution (coarsest level)**.
  - **Upsample the flow** and refine it at each level until reaching full resolution.
- This allows the algorithm to **capture large displacements** at low resolution and refine

Reduce the resolution!

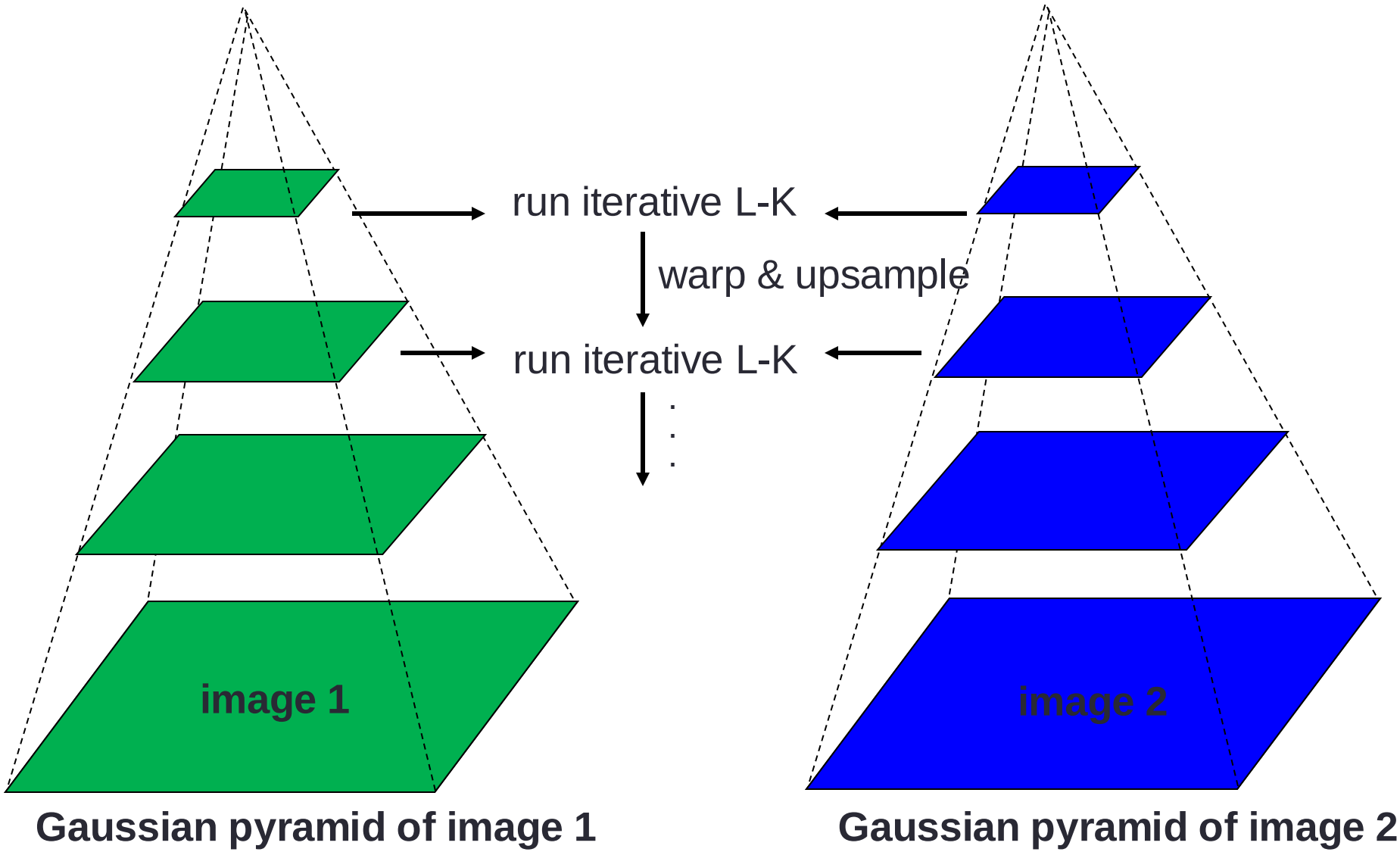




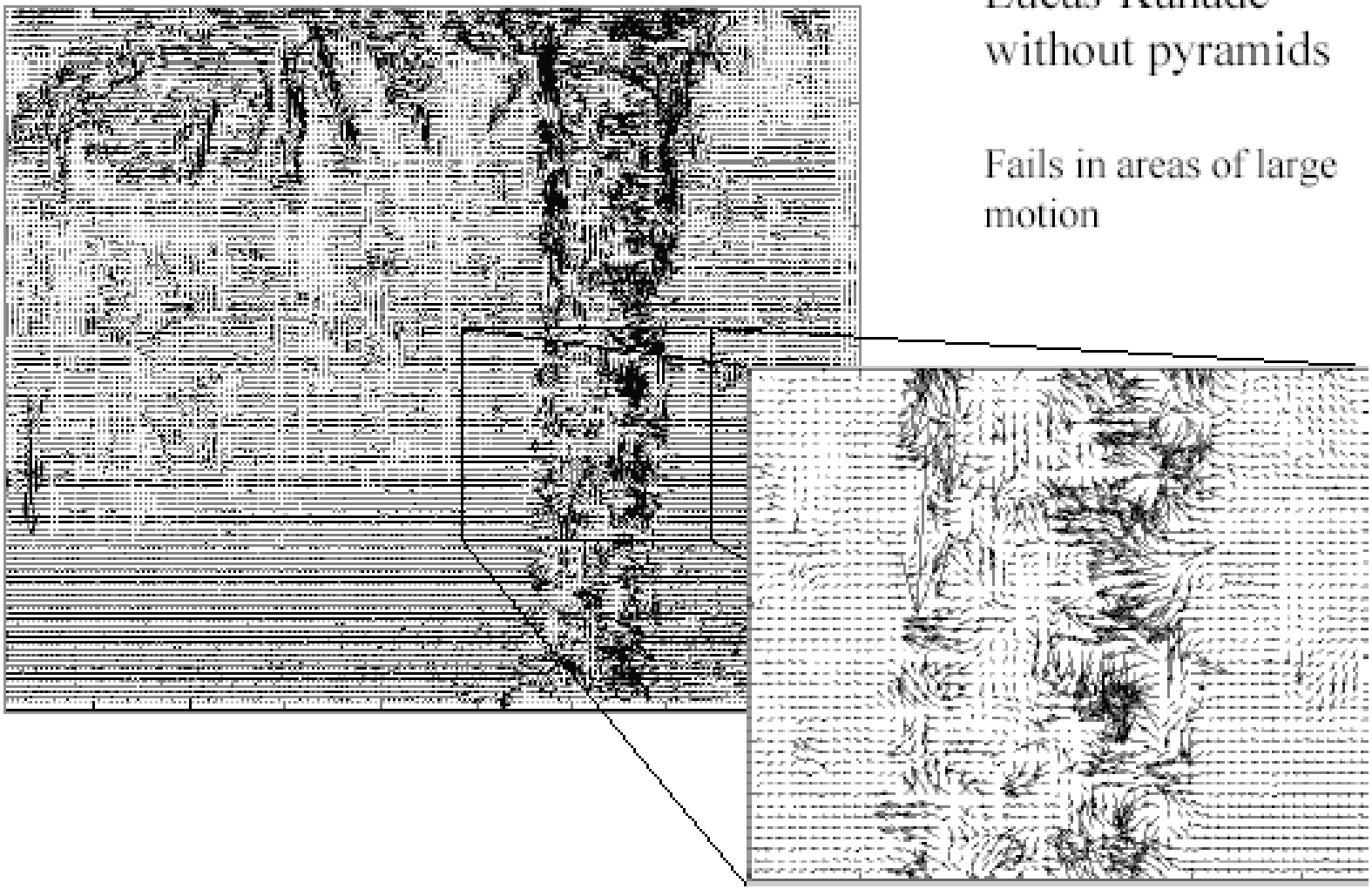
Coarse-to-fine optical flow estimation



Coarse-to-fine optical flow estimation

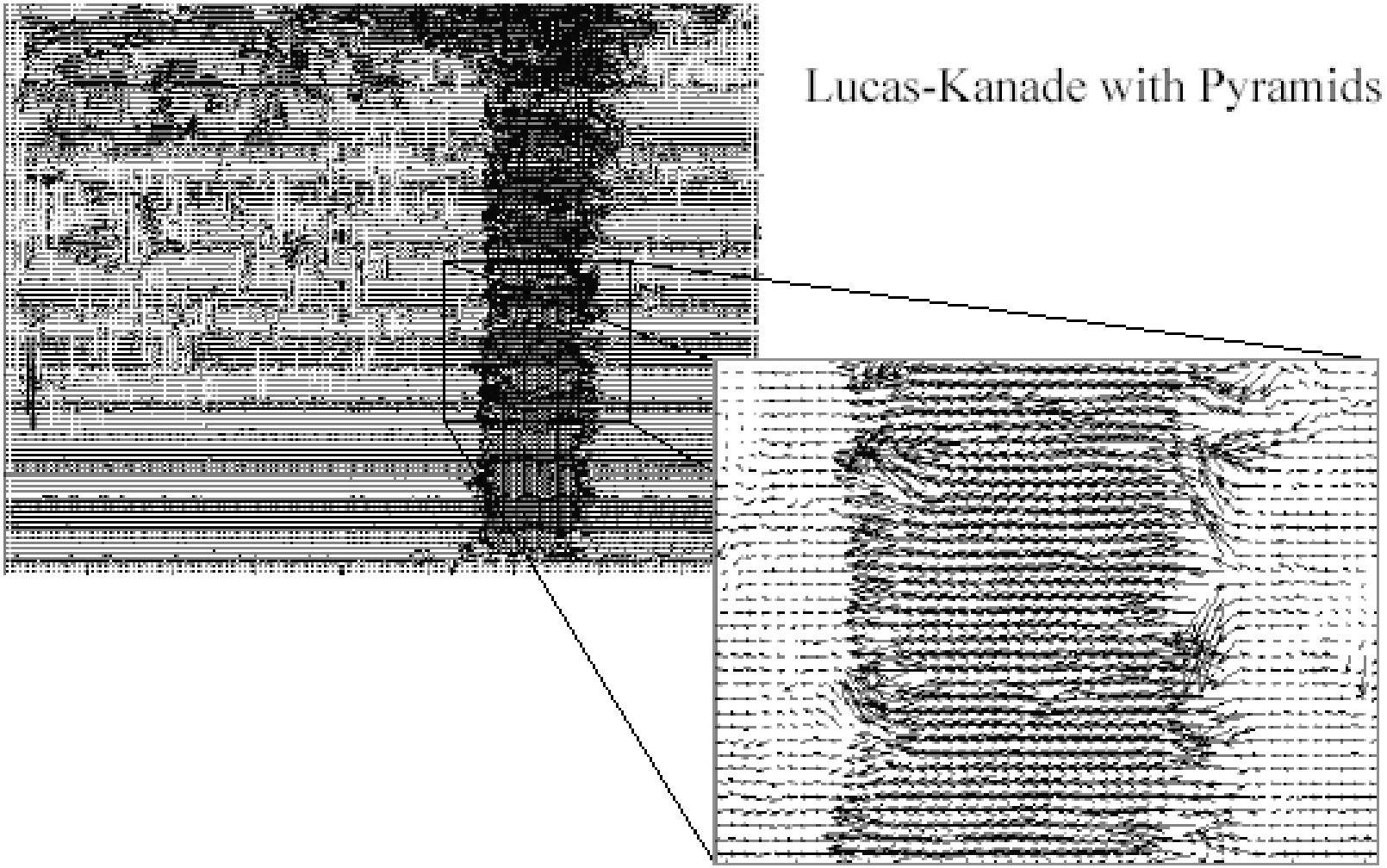


Optical Flow Results



* From Khurram Hassan-Shafique CAP5415 Computer Vision 2003

Optical Flow Results



* From Khurram Hassan-Shafique CAP5415 Computer Vision 2003